

Lighter, Stronger, Optimized

Optimization and System Identification Applications

Ihar Antonau (Technical University of Braunschweig)



Overview

- Optimization Application
 - Motivation
 - Introduction
 - Key features
 - Example
- System Identification Application
 - Motivation
 - Introduction
 - Key features
 - Examples



Optimization App

6/11/2024 – 8/11/2024

Kratos Workshop 2024 - Padova



What is it for?

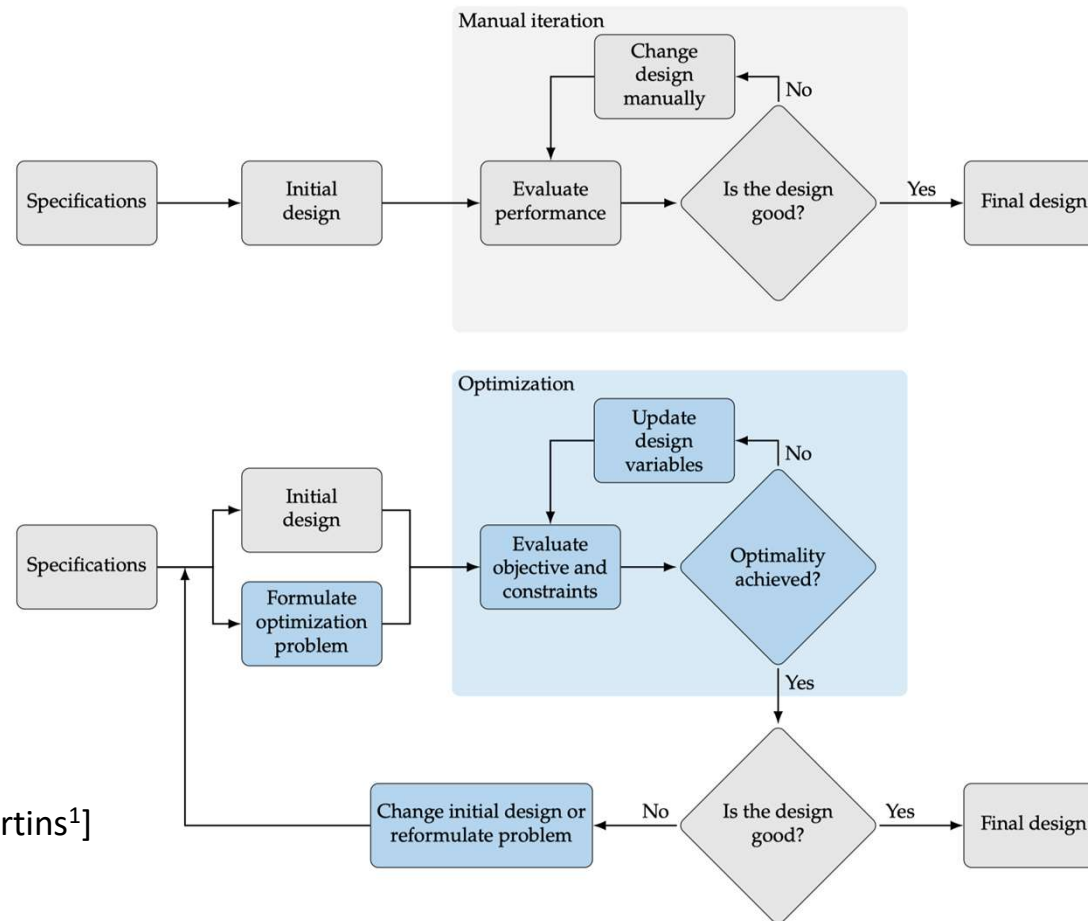


Fig. Conventional (top) versus design optimization (bottom) process, [Martins¹]



Why do we need a new OptApp?

- Have general algorithm interface to allow easy coupling with external tools.
- Develop general purpose optimization application, that can solve various engineering optimization problems with various design parameters: Shape, Topology, Thickness...
- Be Modular to allow different parameterization techniques, design variables scaling, special features.



Optimization Problem

$$\begin{aligned} & \text{minimize : } f(\mathbf{x}) \\ & \text{design variables : } \mathbf{x} \\ & x_i^l \leq x_i \leq x_i^u \\ & \text{s.t.: } g_j(\mathbf{x}) \leq 0, \text{ where } j = 1..n_g \\ & h_k(\mathbf{x}) = 0, \text{ where } k = 1..n_h \end{aligned}$$

Response functions

Analysis

Controls

Optimization Algorithm

Optimization Problem



Most common Optimizer Interface

SciPy Example

```
>>> import numpy as np
>>> from scipy.optimize import minimize
```

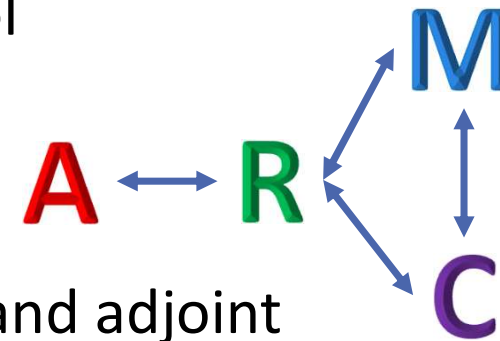
```
>>> def rosen(x):
...     """The Rosenbrock function"""
...     return sum(100.0*(x[1:]-x[:-1]**2.0)**2.0 + (1-x[:-1])**2.0)
```

```
>>> x0 = np.array([1.3, 0.7, 0.8, 1.9, 1.2])
>>> res = minimize(rosen, x0, method='nelder-mead',
...               options={'xatol': 1e-8, 'disp': True})
Optimization terminated successfully.
    Current function value: 0.000000
    Iterations: 339
    Function evaluations: 571
```

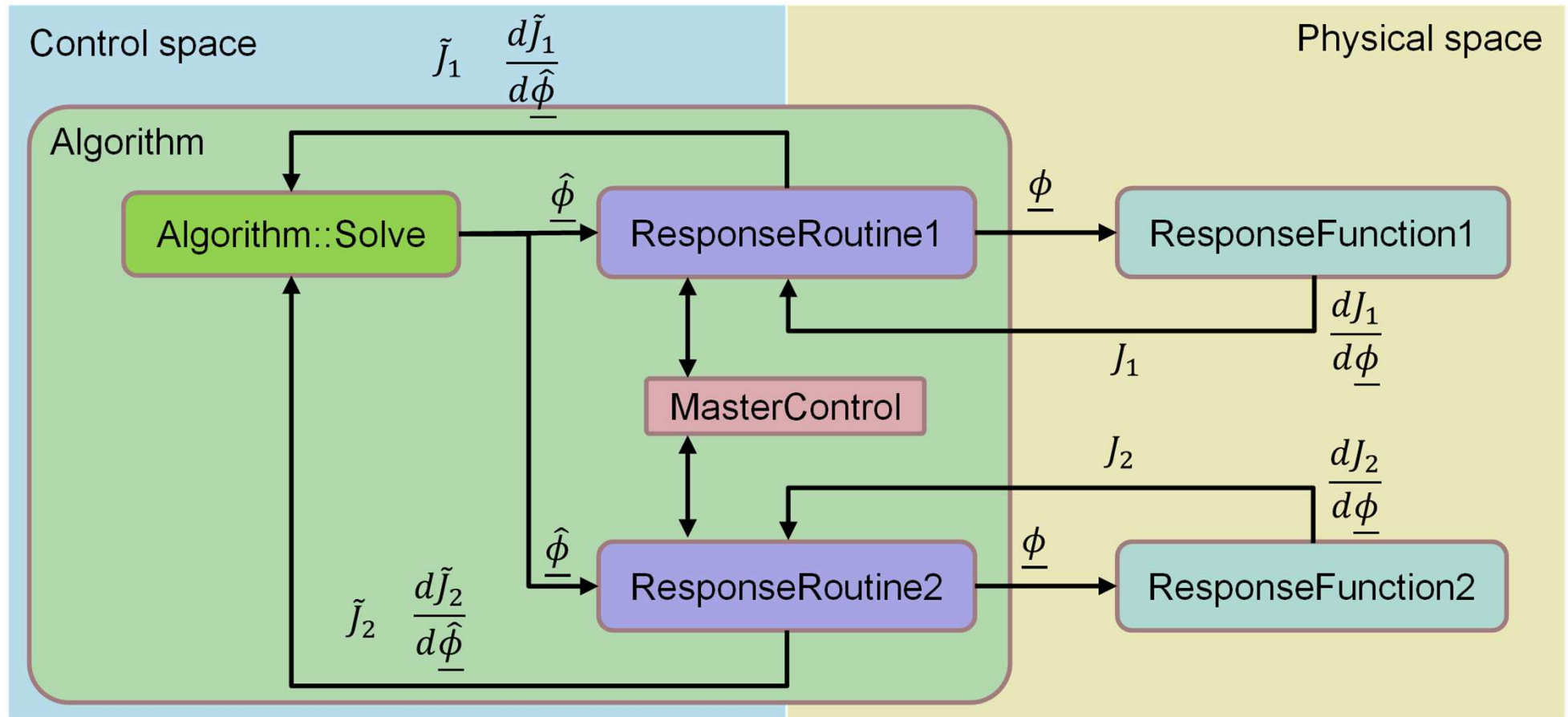


Main workflow

- Algorithm connect only with Response Functions like, $f(x)$, where x is a vector of design variables;
- Response Routine (Response) calls the control to update the FE model with new design parameters;
- Control updates the Model
- Response call Model to solve PDEs of primal and adjoint problem
- That simplifies the algorithm logic and interface allowing easier coupling with external optimizers (SciPy, PyRol, Nlopt)

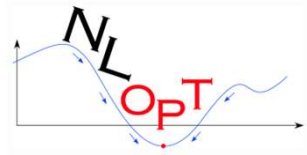


Algorithm – Response relation

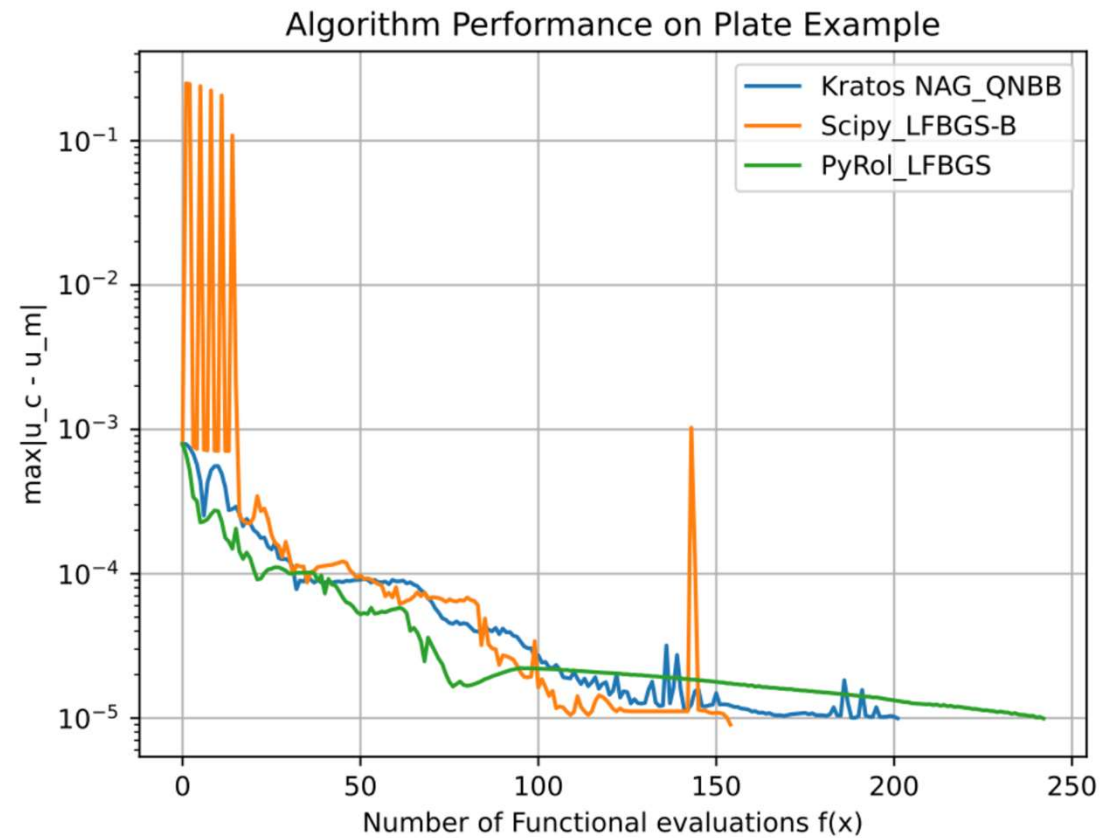
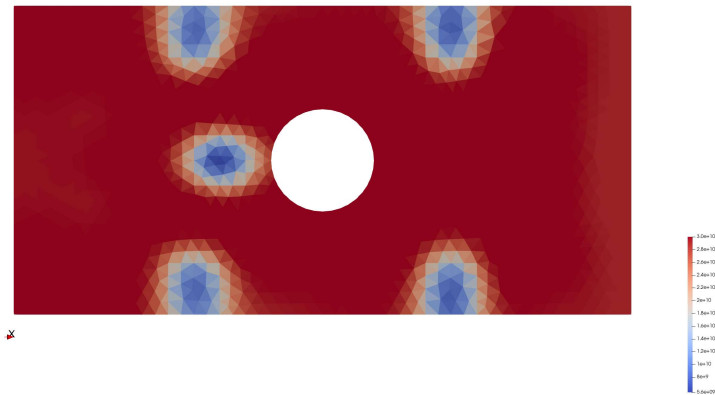
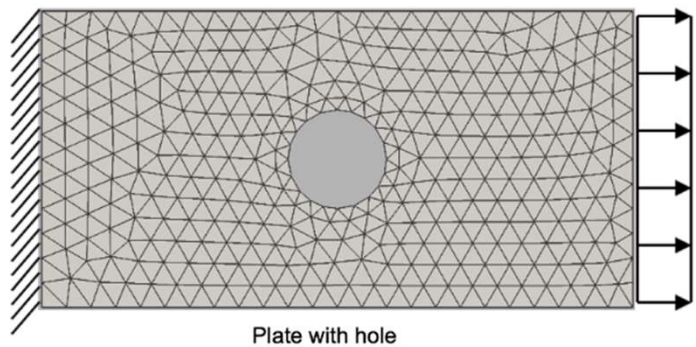


Algorithms in Kratos

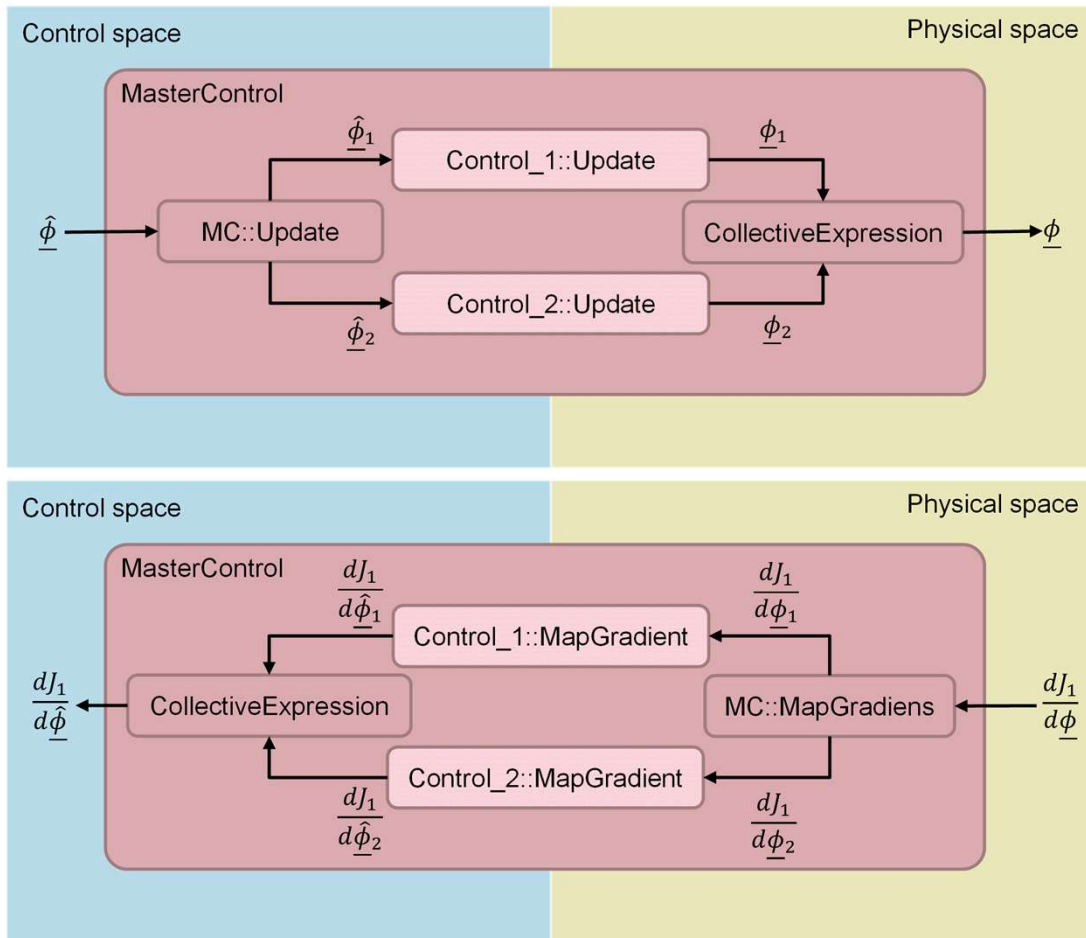
- Inhouse codes:
 - Steepest descent
 - Gradient Projection and Relaxed Gradient Projection
 - Nesterov Acceleration Gradient
- External Libraries
 - SciPy
 - NLOpt
 - PyRol
 - Provides huge library of opt. algorithms + optimization problem property tests



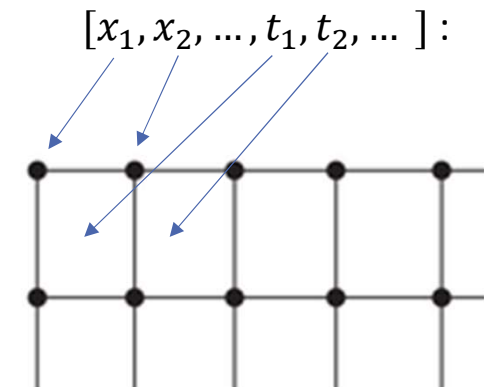
Example: Convergence study for SI problems



How to handle various design parameters?



Where all design variable are stored in a **collective expression!!!**



For algorithm it is just a numpy vector, where all design parameters are connected with the model parameters.



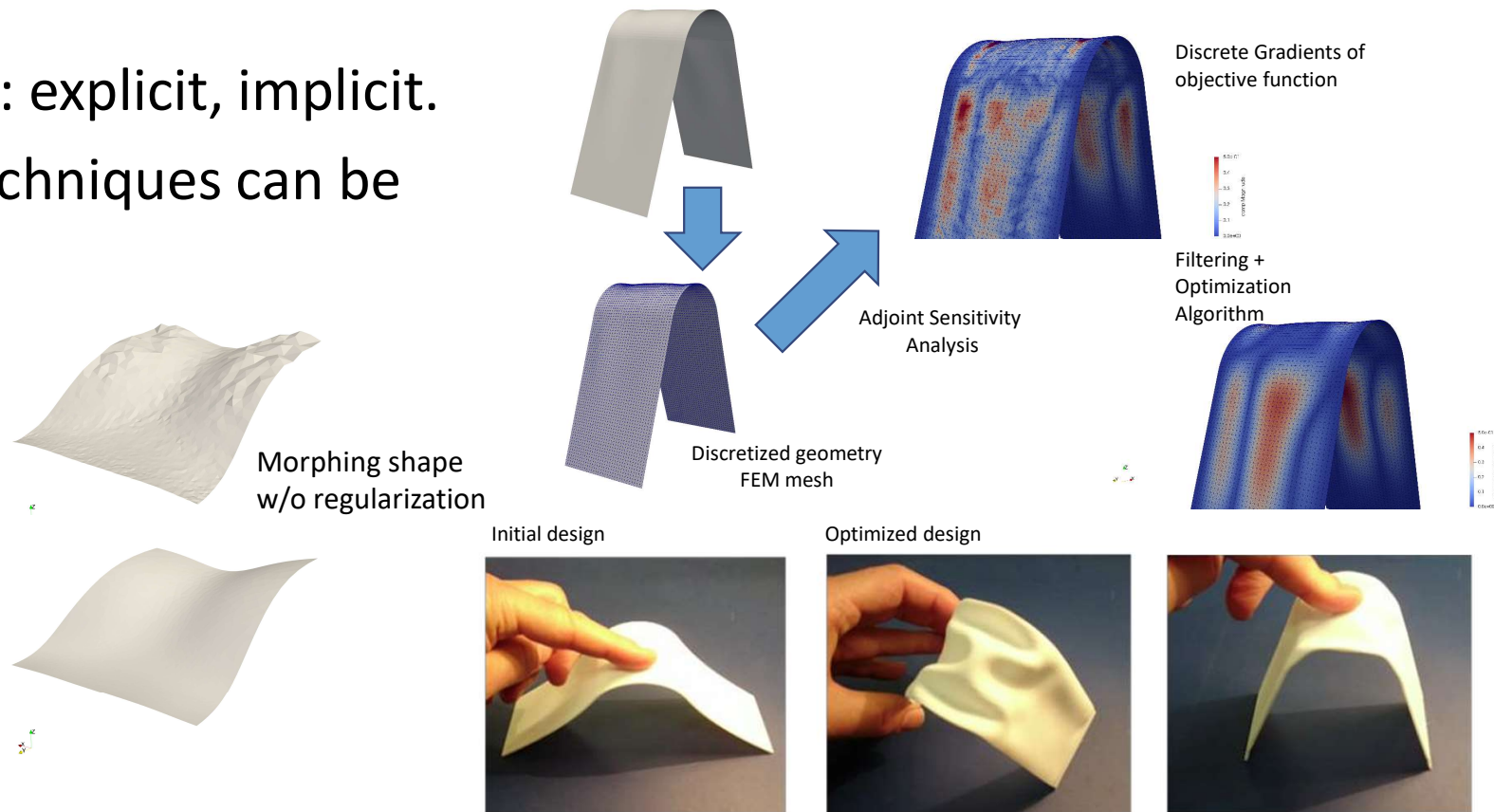
Modularity

- Generalized controls, like: shape, material.
- Different techniques can be applied to it, for instance: des. Variable projection, filtering, damping, etc.
- Different conv. criteria, line search can be applied almost independently
- Response functions can be from different applications, for instance, System Identification.



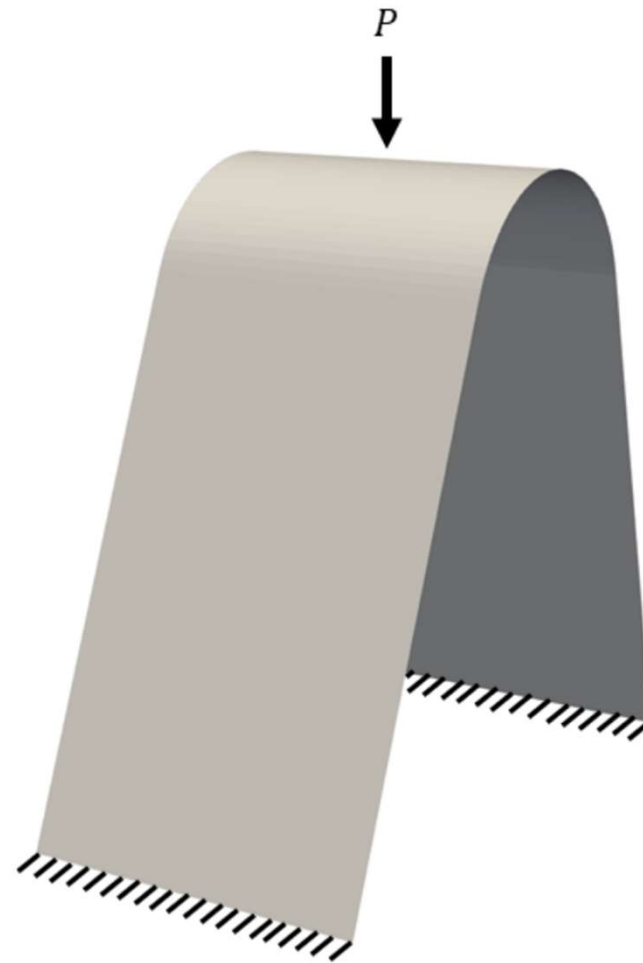
Shape Optimization

- Vertex Morphing: explicit, implicit.
- Other filtering techniques can be easily added
- Boundary cond.
- Volume mesh motion

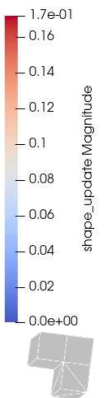
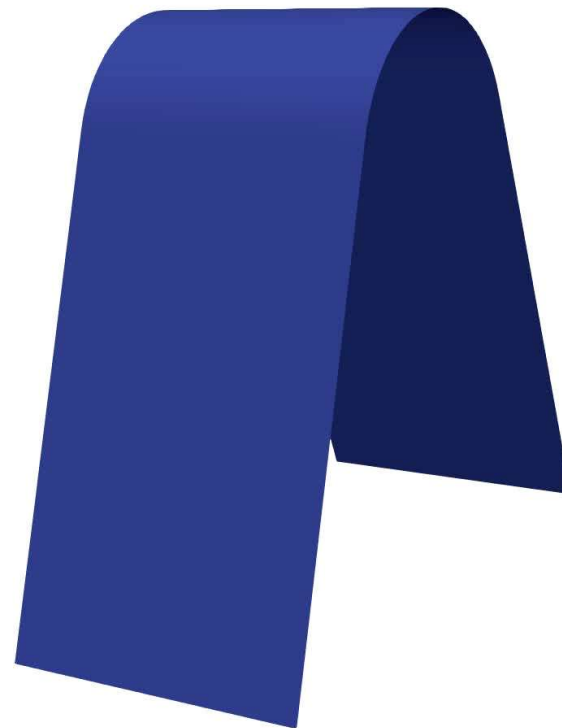
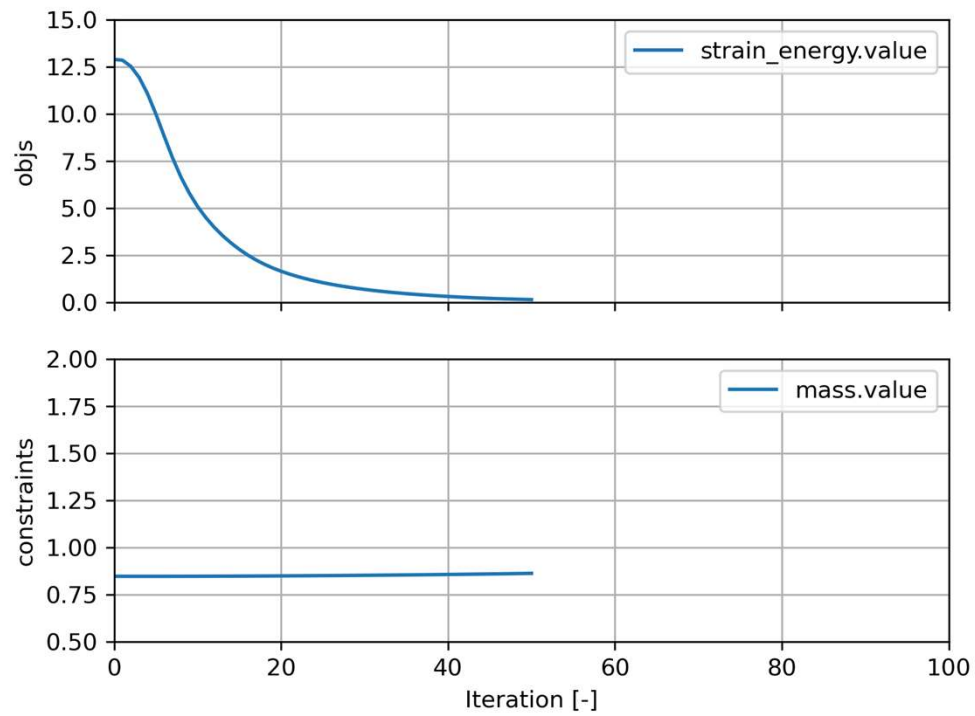


Example: 3D Shell

Min: strain energy
s.t. mass ≤ 1.2 kg

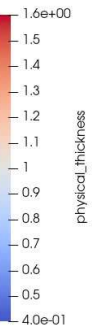
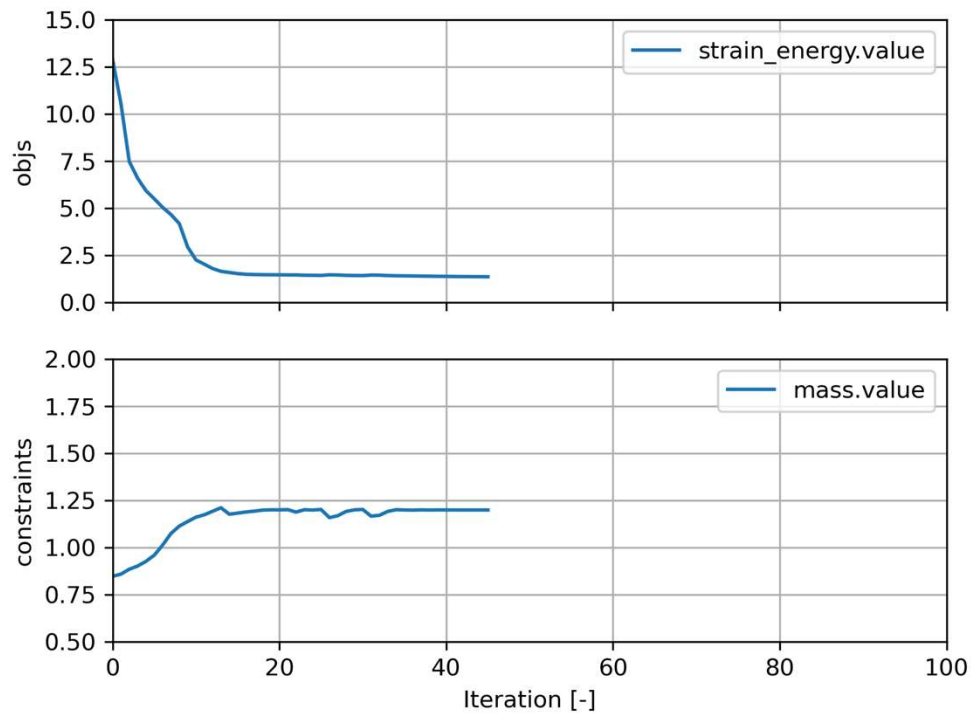


Shape



Thickness

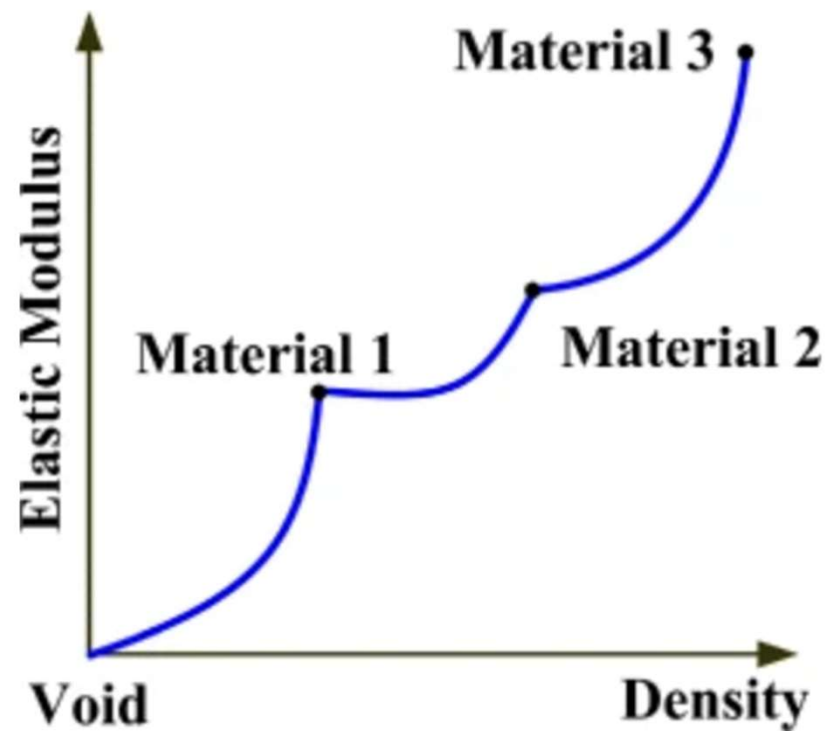
Allowing multiple thicknesses: {0.16, 1.0, 0.4}



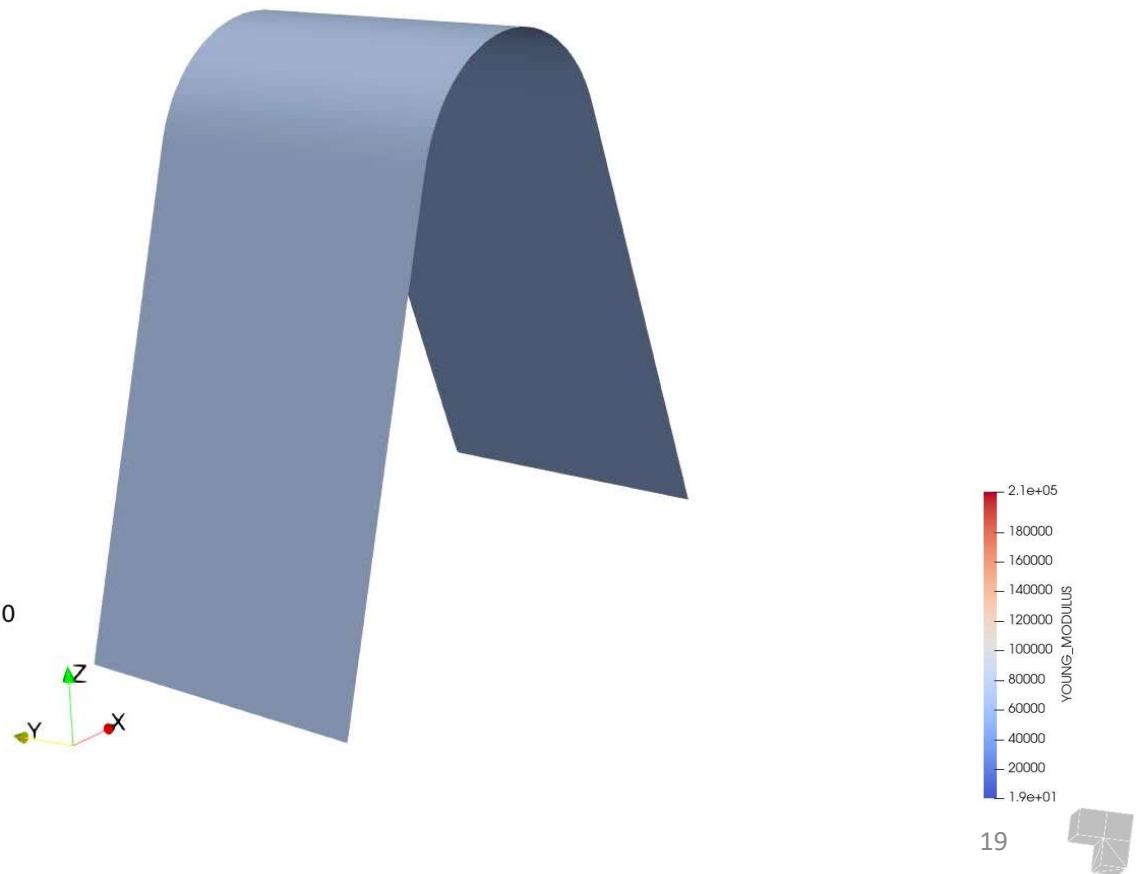
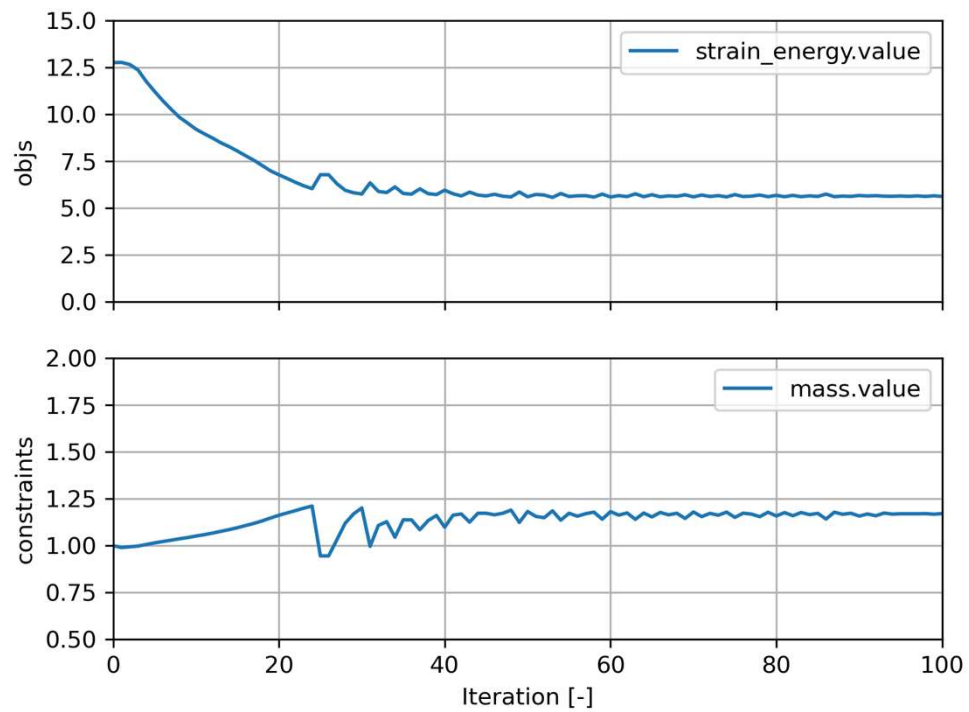
Material

Allowing multiple materials:

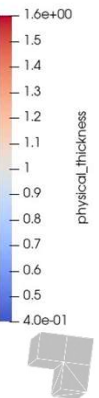
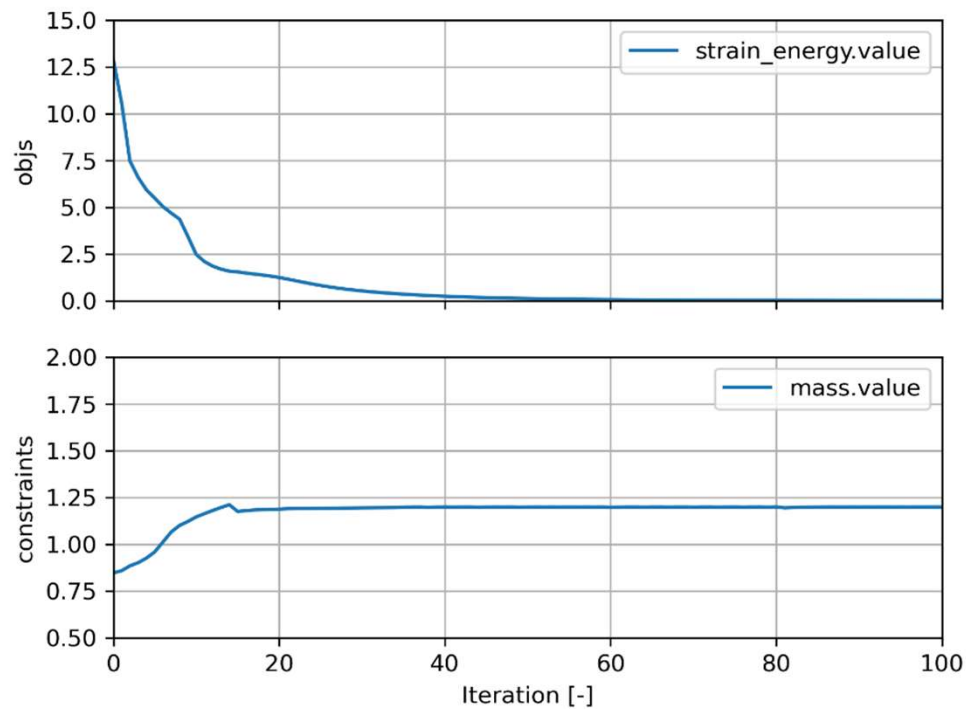
- Steel
- Aluminum
- Void
- ...



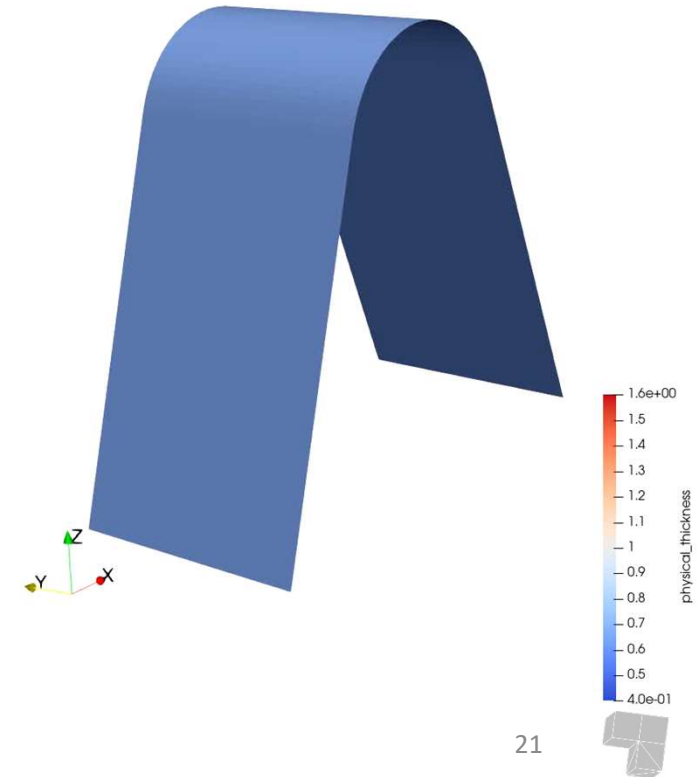
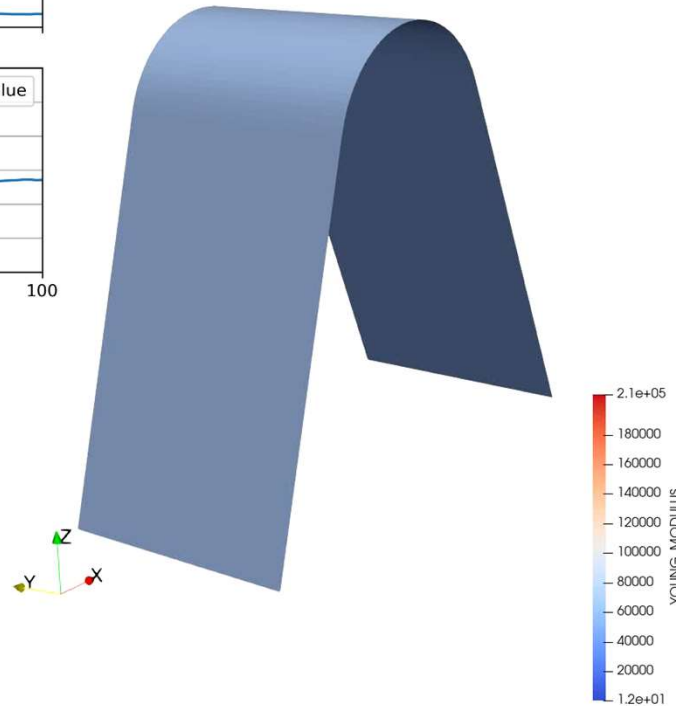
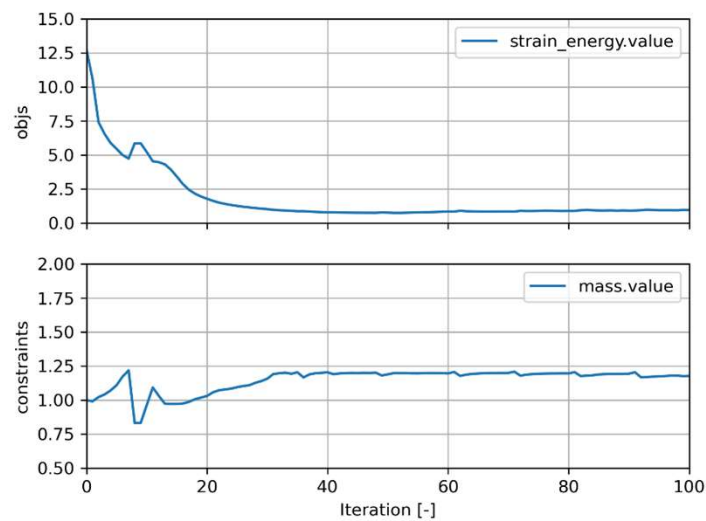
Material



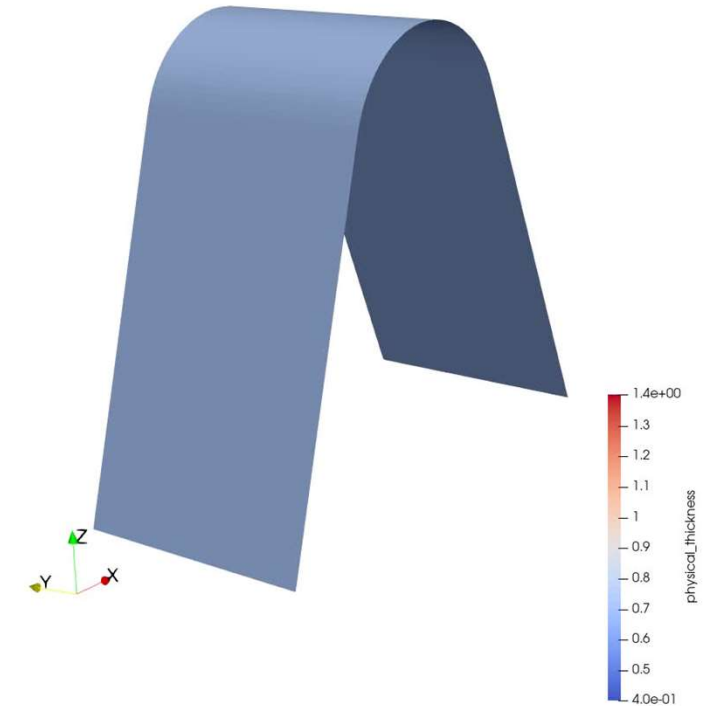
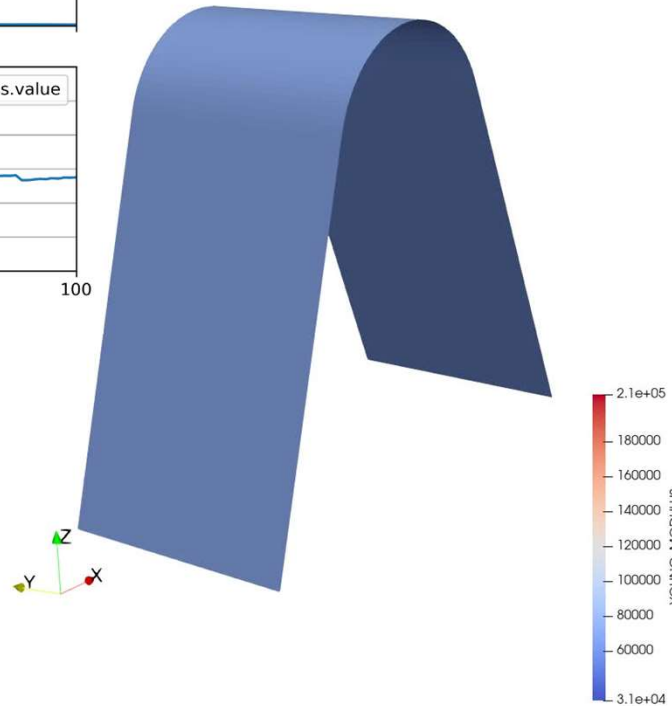
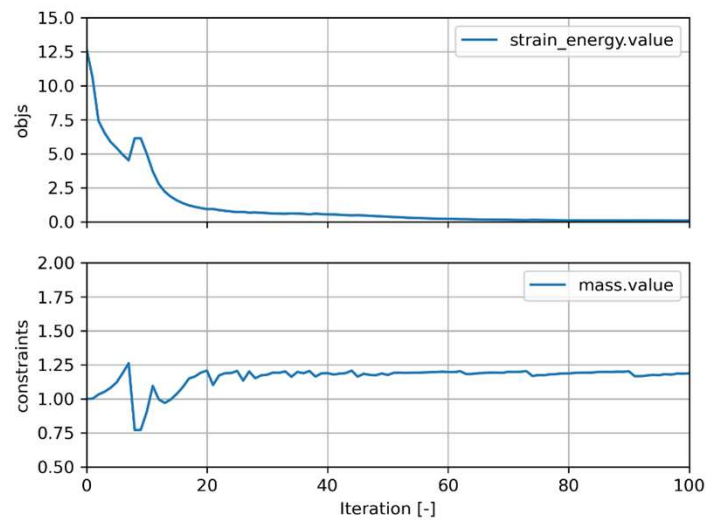
Mixed variables: Shape + Thickness



Mixed variables: Thickness + Material



Mixed variables: Thickness + Material + Shape

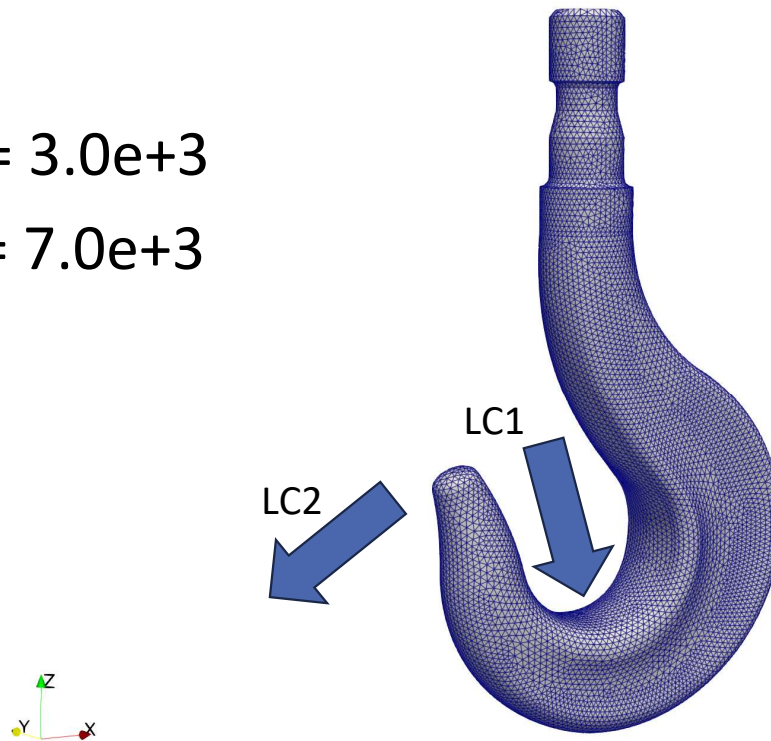


Example: 3D Solid Hook

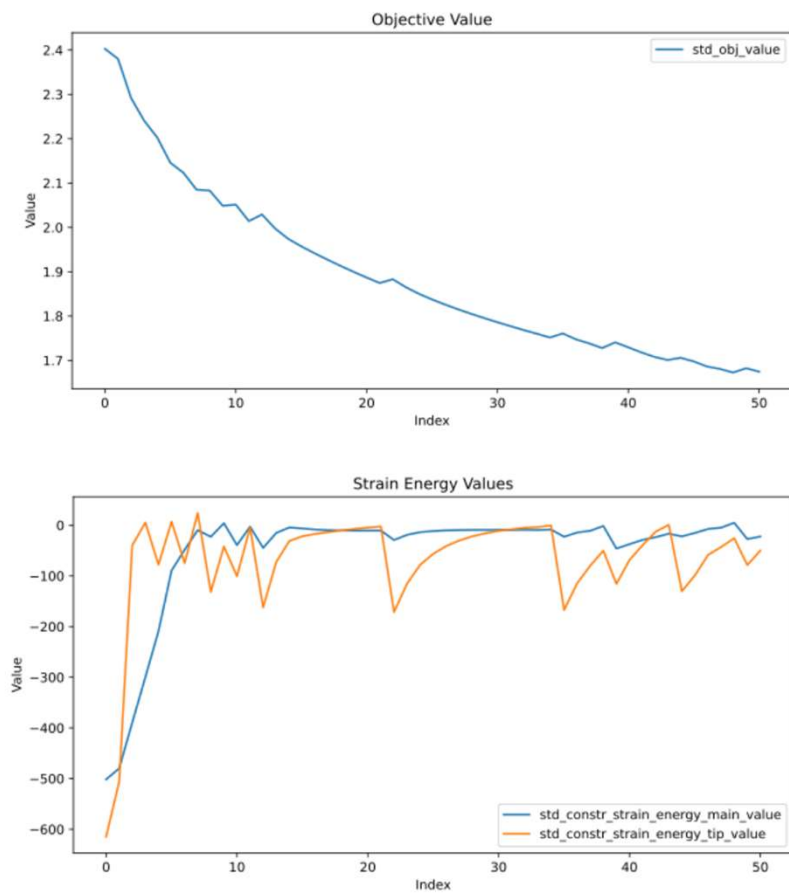
Min: mass

s.t. LC1: strain_energy $\leq 3.0e+3$

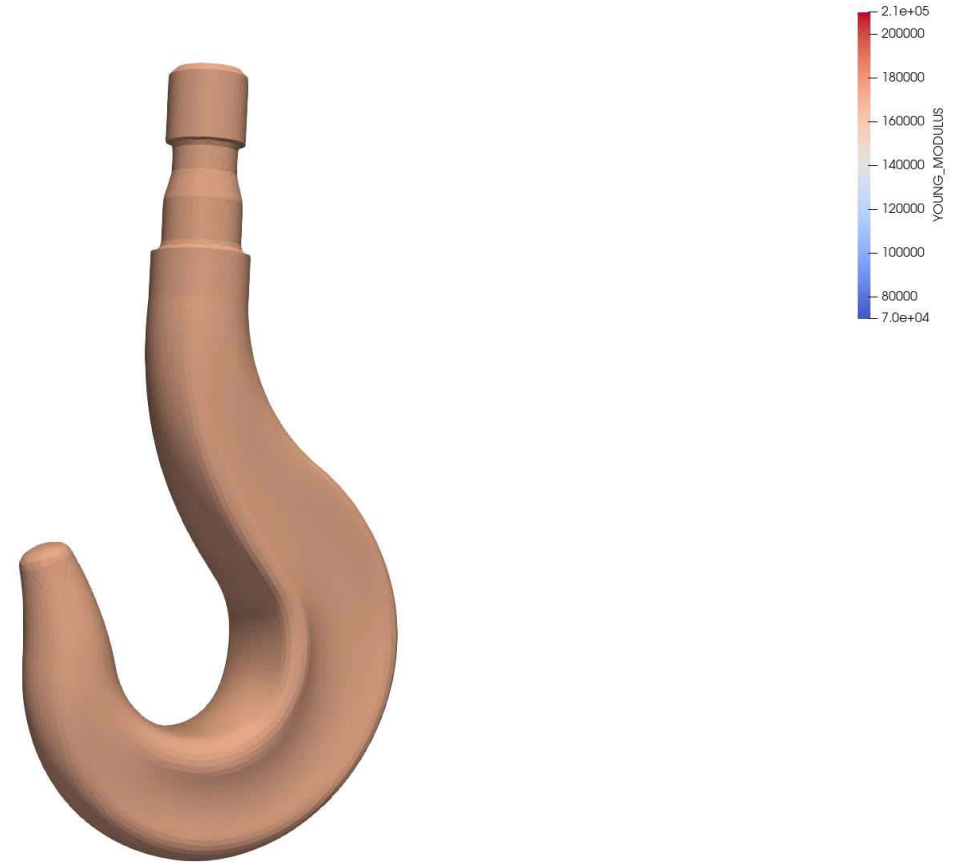
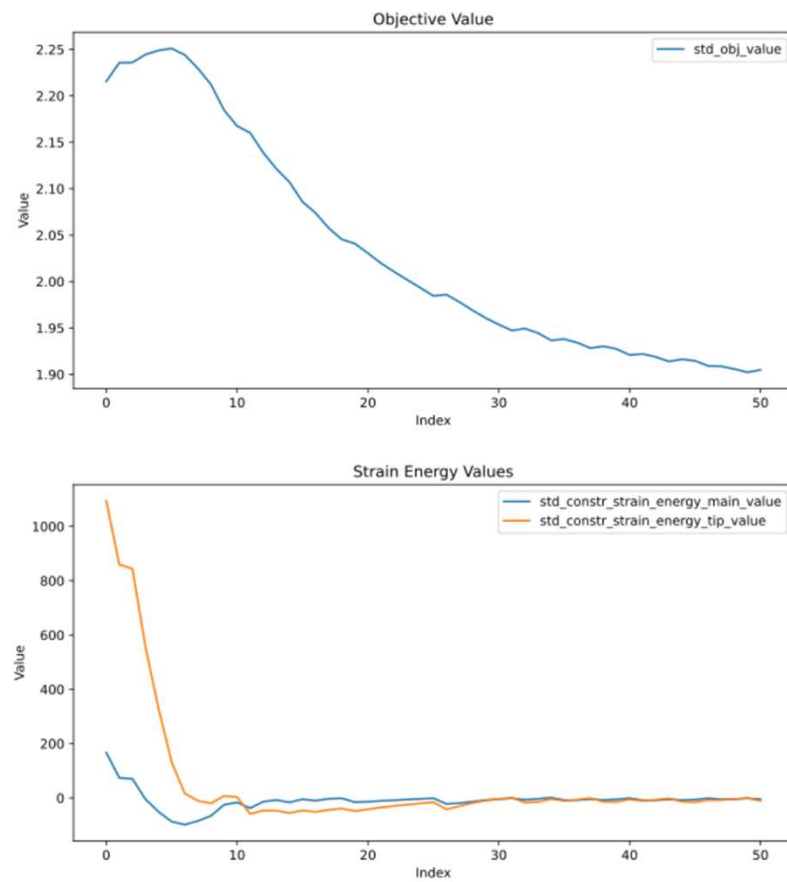
LC2: strain_energy $\leq 7.0e+3$



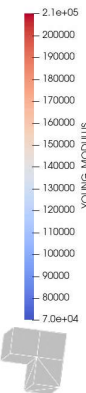
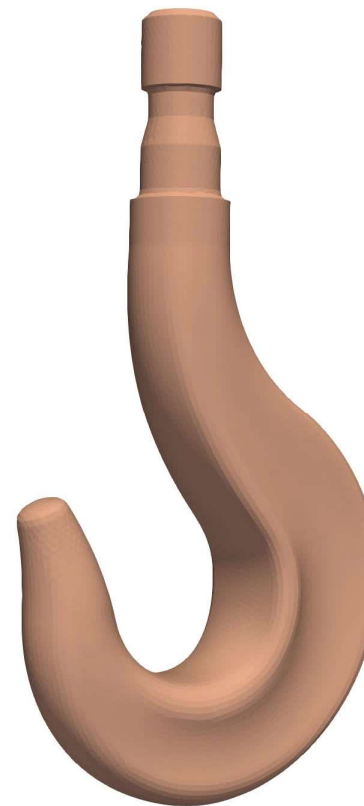
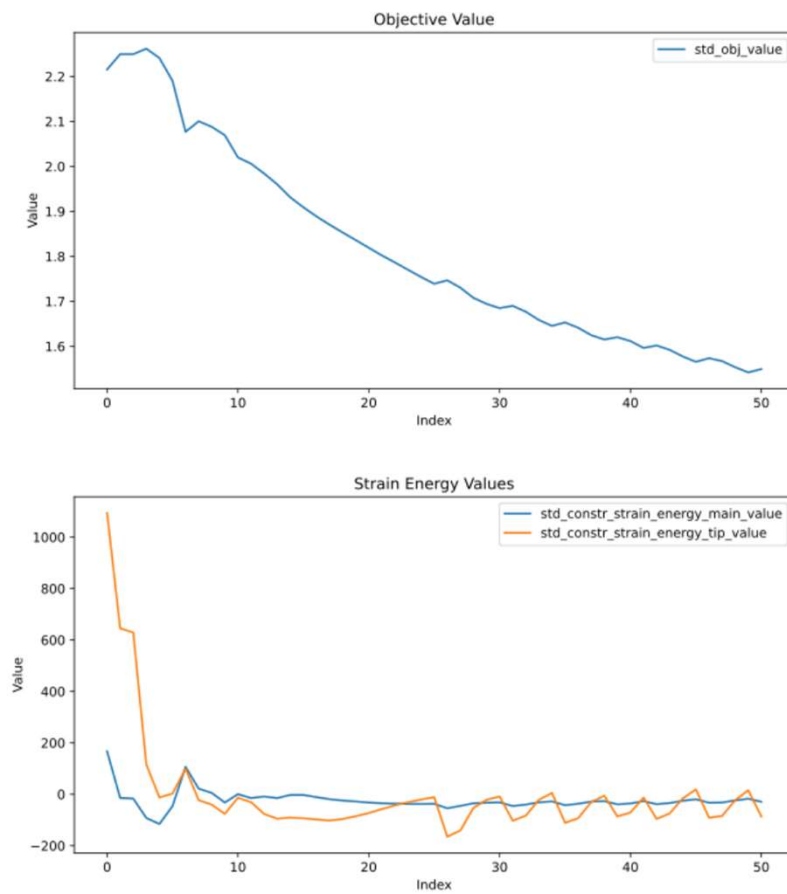
Shape



Material



Shape + Material



Summary

- Good collection of optimization algorithms
- Support mixed types of design variables
- Extensibility
- Modularity & Flexibility
- Can be used as a benchmark platform for Engineering Design
- Making the OptApp -> Physics agnostics
- ToDo: Refactoring and simplifying adjoint formulation

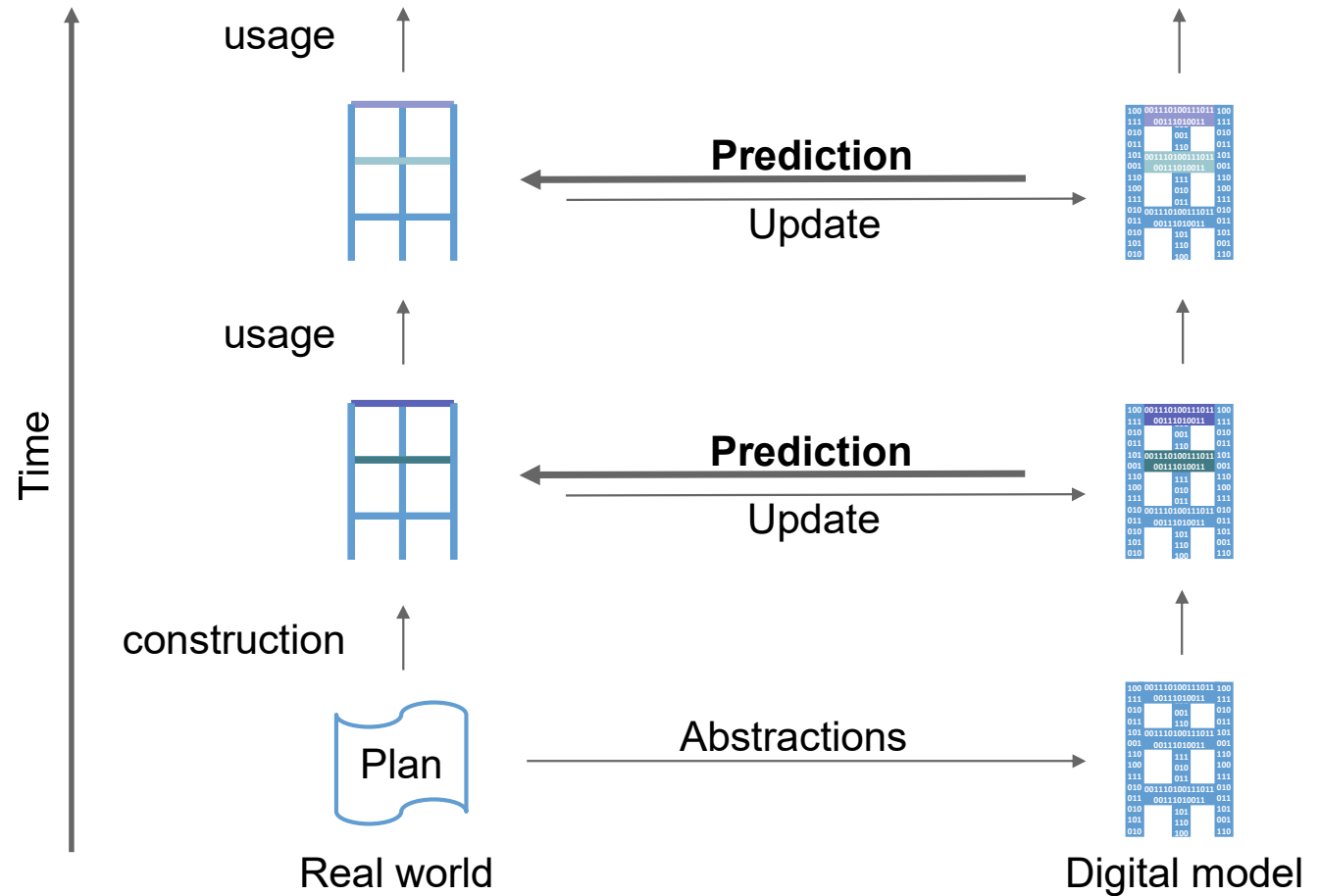


System Identification App



What is it about?

Digital twin

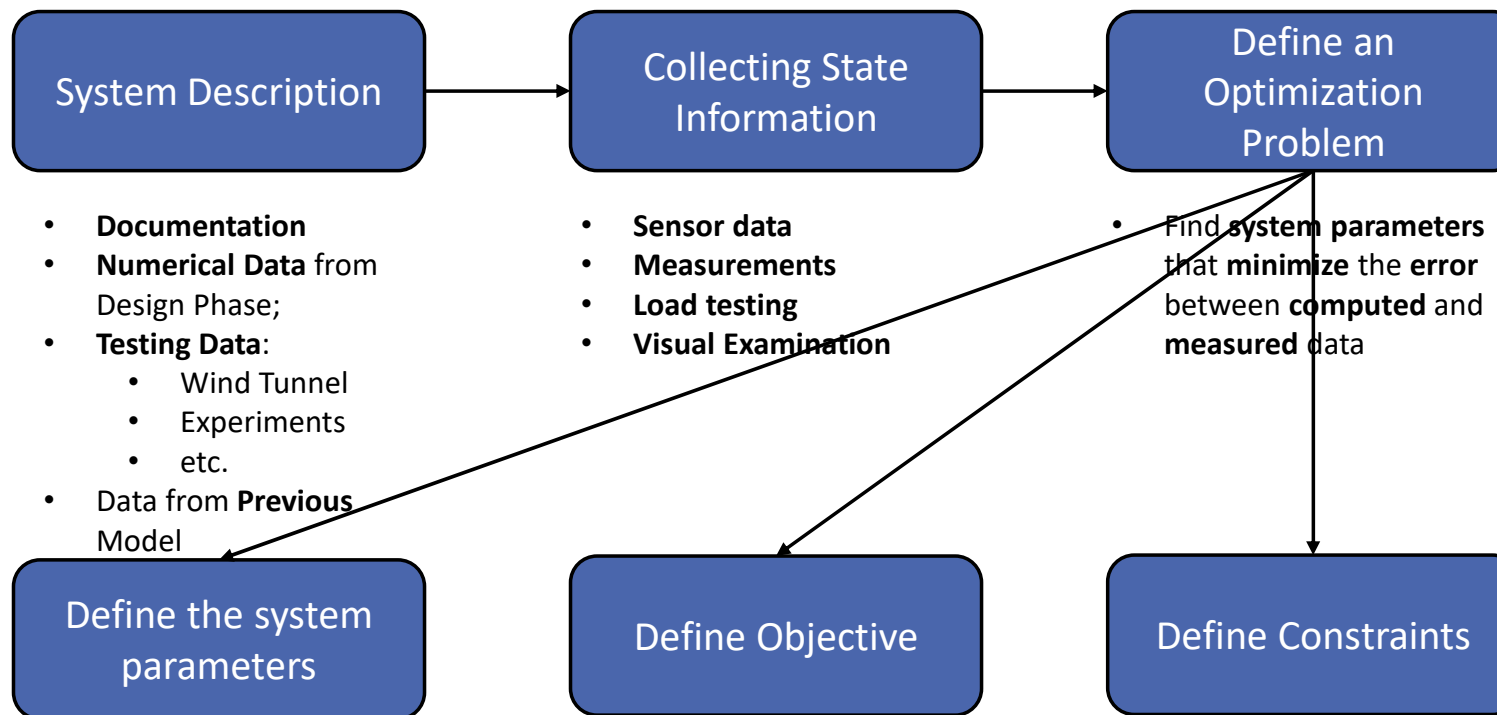


Motivation

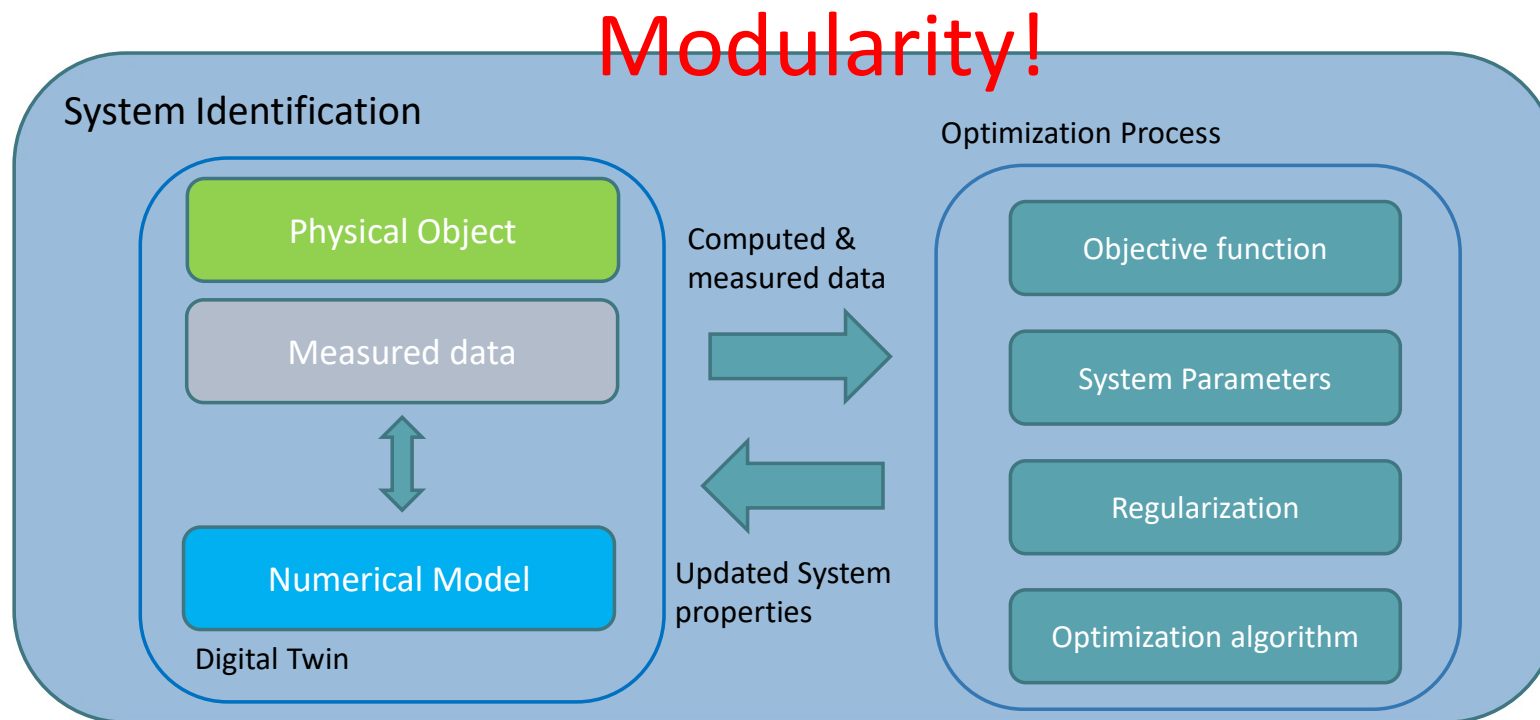
- How does the structure performance change through time?
- How to find weaknesses in the structures from the measured data?
- Where to place the sensors to monitor the structure?
- How to formulate and solve this problems?



System Identification as an Optimization Problem

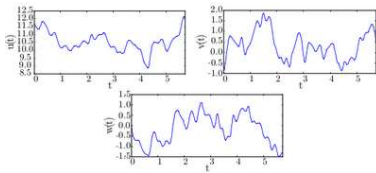


Determining system parameters via optimization



Why Modularity?

Multiload cases



Various damage scale / localization



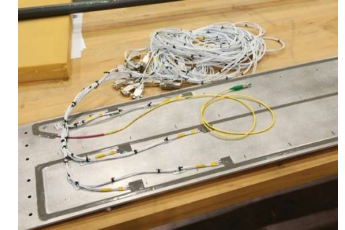
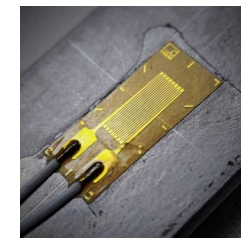
Meso-scale phenomenon



Different measured data & sensor types

- ▲ Anemometers
- ◆ 3D Accelerometers
- ✱ Temperature Sensors
- Dynamic Strain Gauges
- + Static Strain Gauges
- Global Positioning Systems
- ↓ Tiltmeters
- Buffer Sensors
- Bearing Sensors
- Electro-Magnetic Sensors
- Barometers, Rainfall Gauges & Hygrometers
- ✱ Corrosion Cells
- Digital Video Cameras
- DWIMS: Dynamic Weigh-in-motion stations

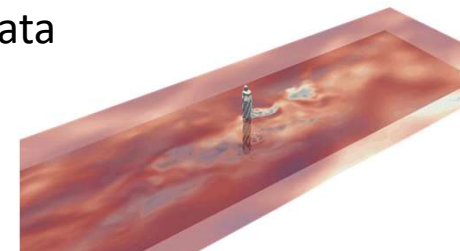
Structural Health Monitoring
www.vackergroup.ae



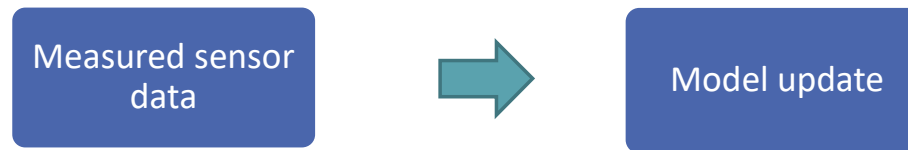
Reference: <https://fprimec.com/sensors-for-structural-health-monitoring/>

Experimental data

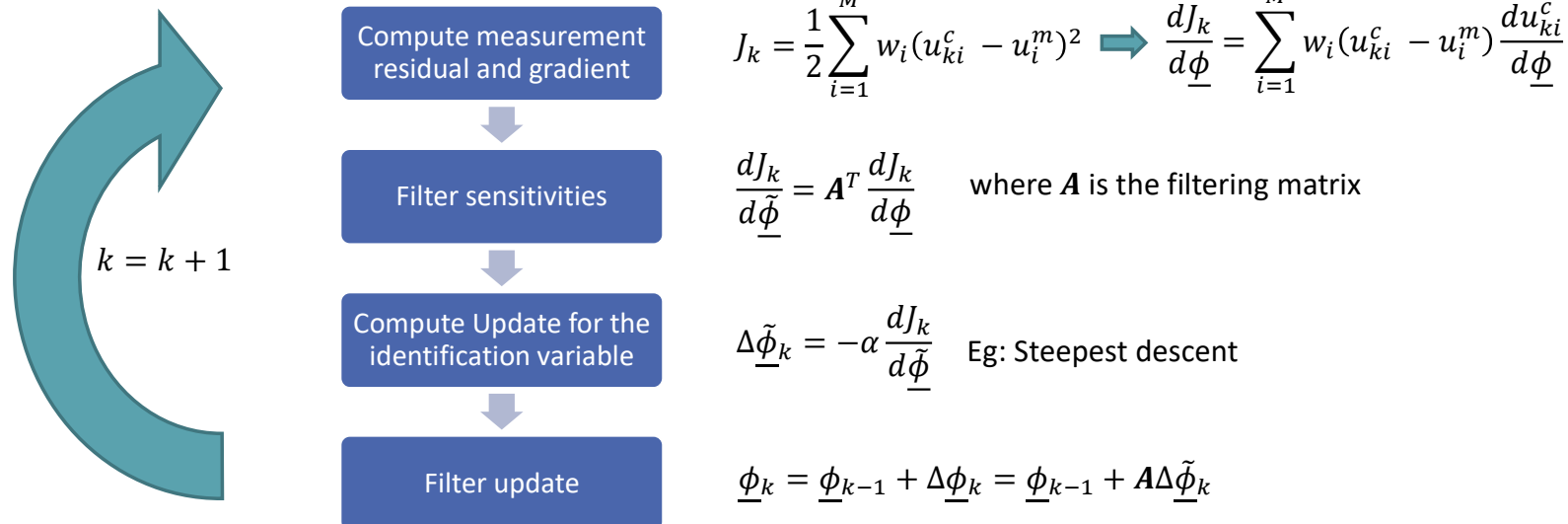
MDA scenarios



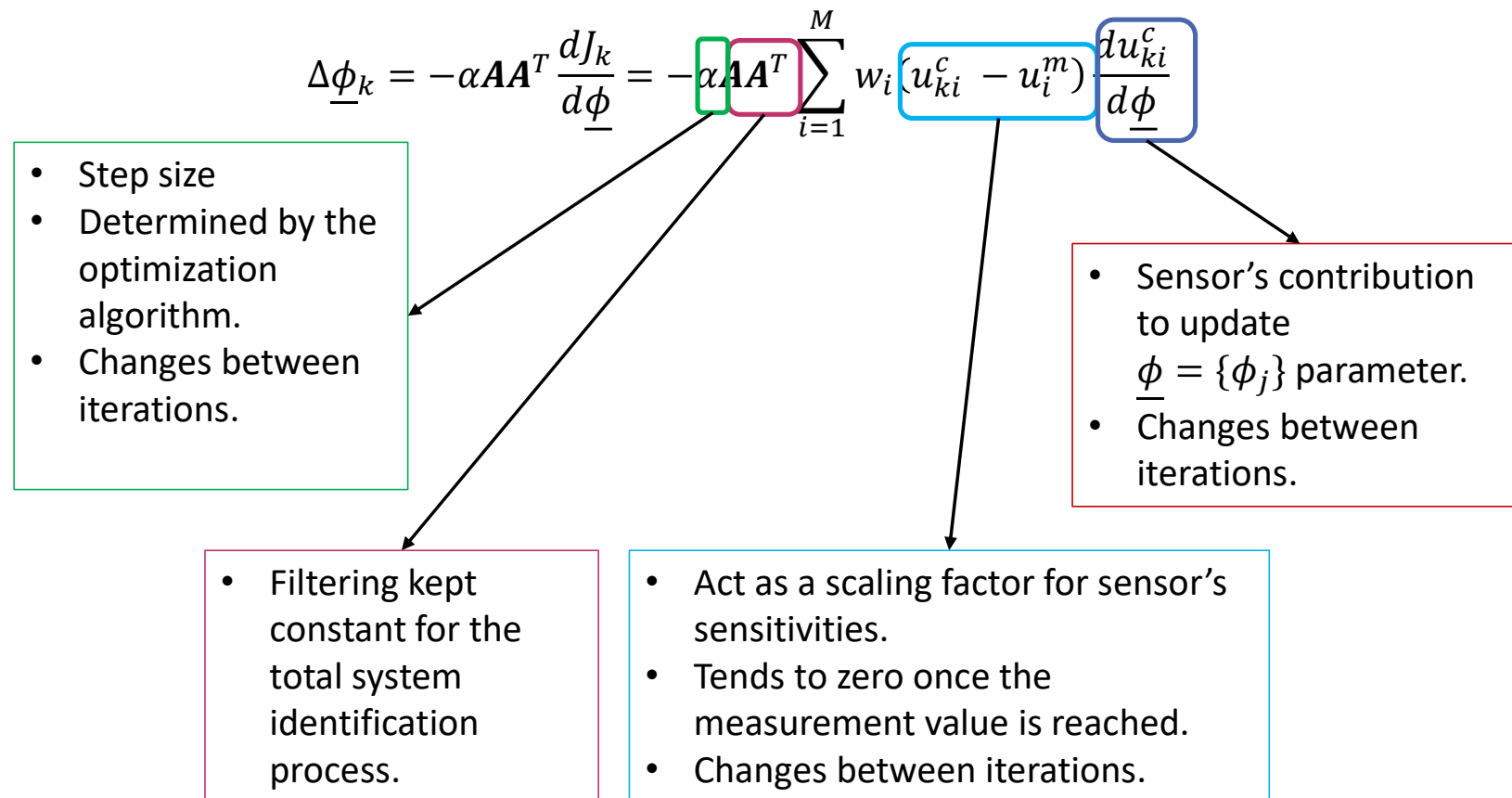
System Identification - How?



- Steps in system identification for M number of sensors at k^{th} identification iteration:



System Identification - How?



2D Plate

- Optimization problem:
- $\min: J(\mathbf{u}, \mathbf{s}) = \frac{1}{2m} \sum_{j=1}^m \omega_j (u_j^{md} - \mathbf{I}_j^d \mathbf{u})^2$
 $m = 55, \omega_j = 1$
- where $\mathbf{u}$ is deformations, $\mathbf{s}$ is Elastic Modulus E at each element.
- Conv. Crit. : max 1e4 iter. or Residual $\leq 1e-10$

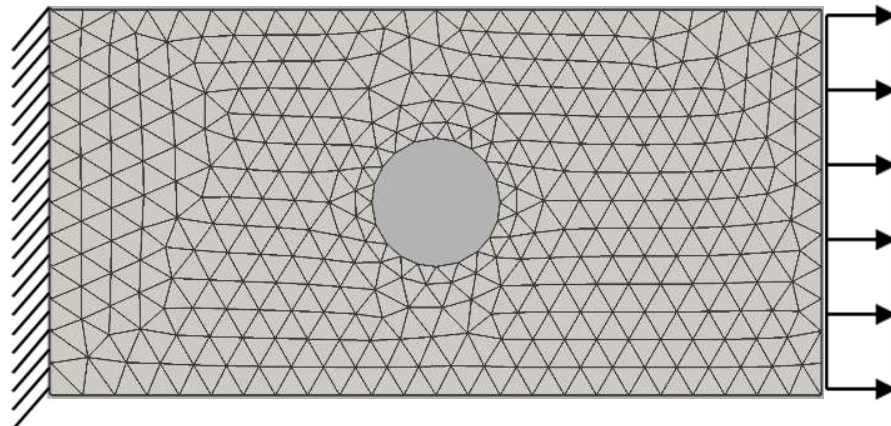
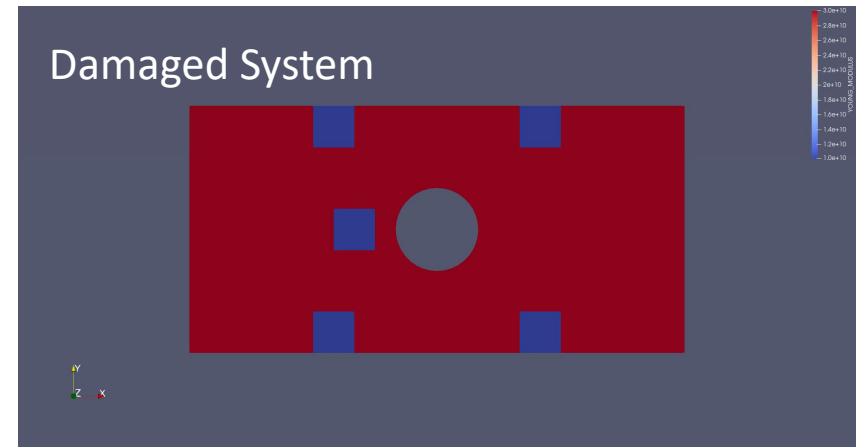
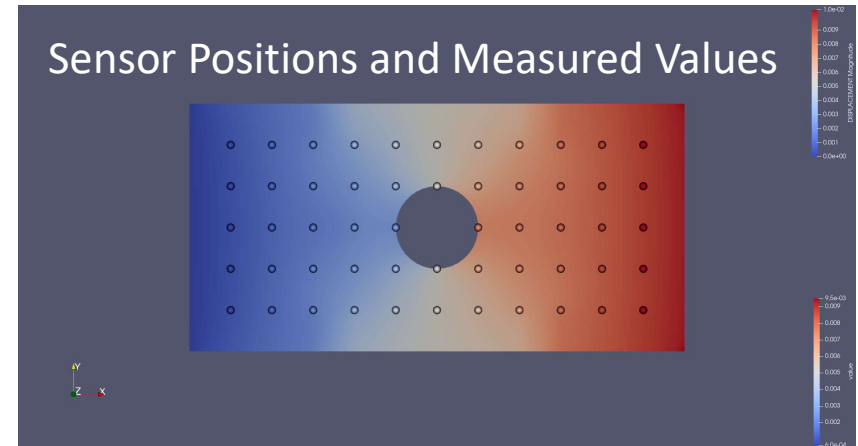


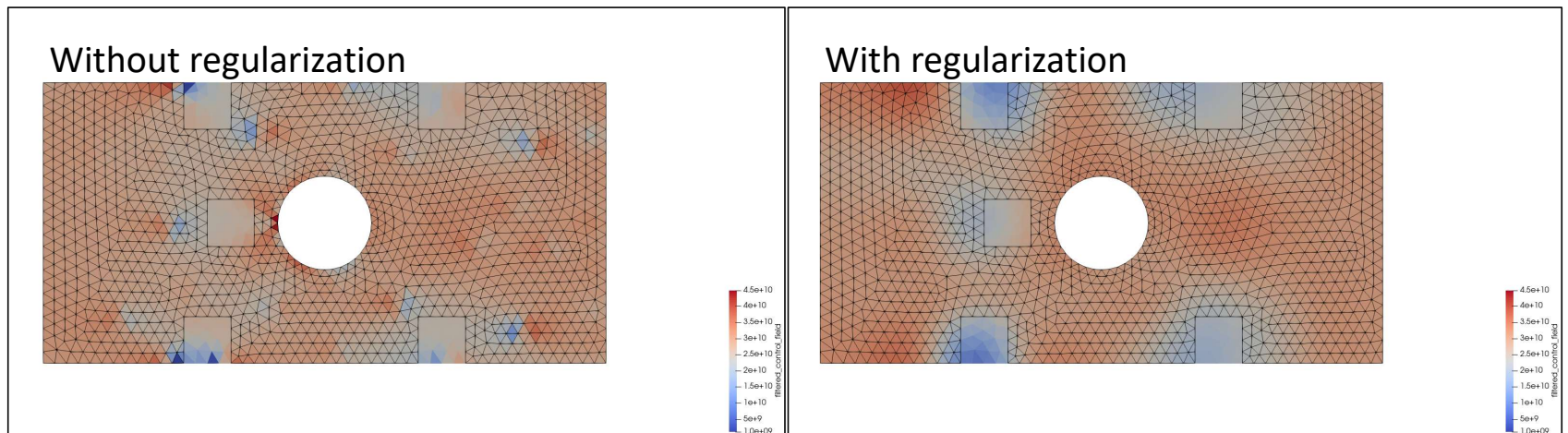
Plate with hole

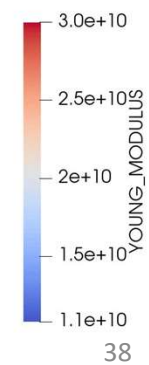
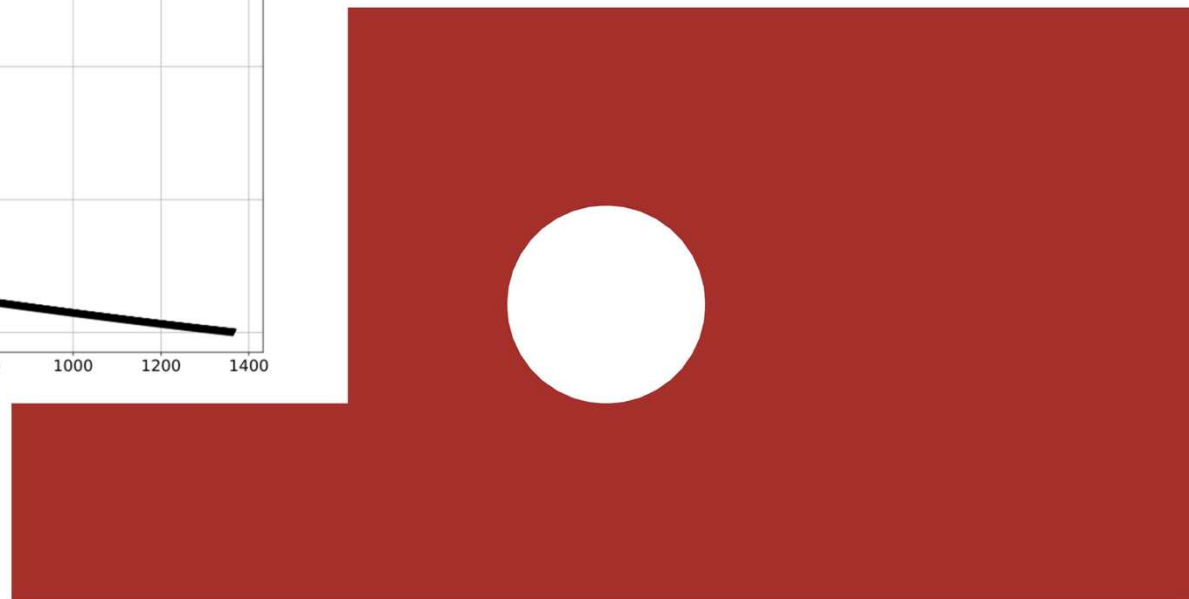
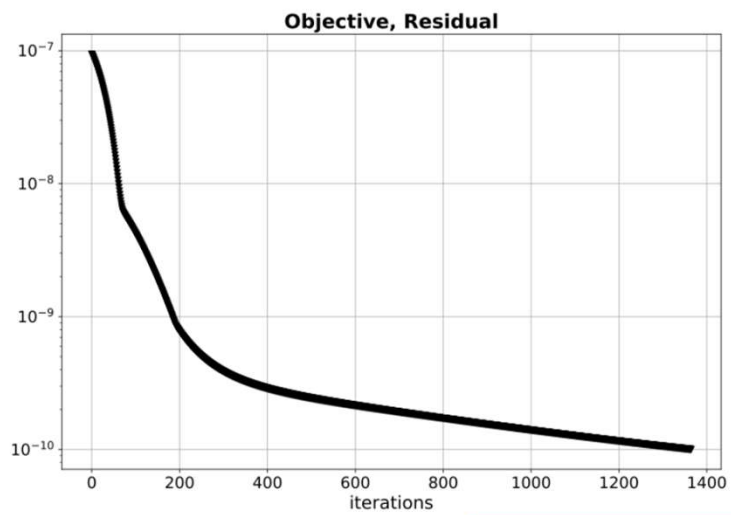


Regularization

Different regularization methods can be applied, which are used in shape and topology optimization:

- 1.(Weak) Laplacian Smoothing: $[1 - \lambda \nabla^2] \mathbf{p} = \mathbf{p}_0$
- 2.Pseudo-Laplacian Smoothing: $[1 - \lambda \nabla h^2 \nabla] \mathbf{p} = \mathbf{p}_0$
- 3.Vertex Morphing: $\mathbf{p} = \int_{\Omega} \mathbf{A}(r) \mathbf{p}_0 d\Omega$





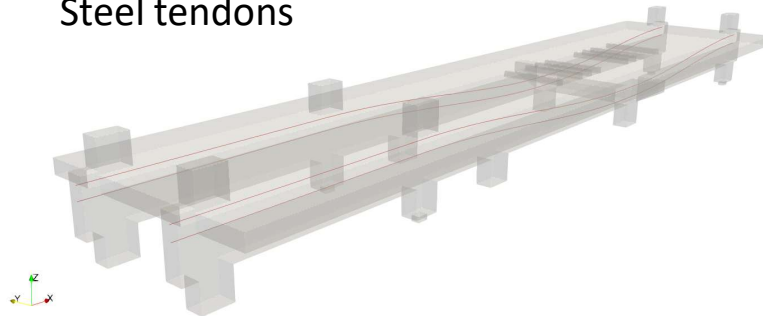
Concrete Bridge

3D high-fidelity **Digital-Twin** of Concrete bridge:

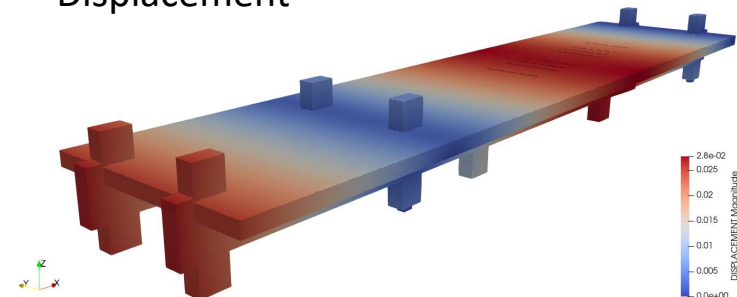
- **multiple** element types:
 - trusses
 - solid small displacement
- **77k unknown** parameters



Steel tendons

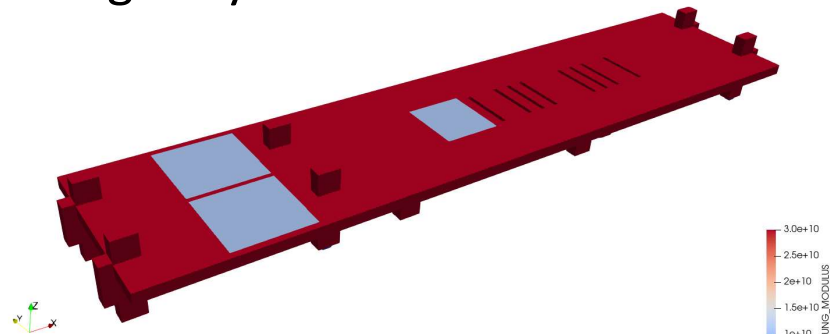


Displacement

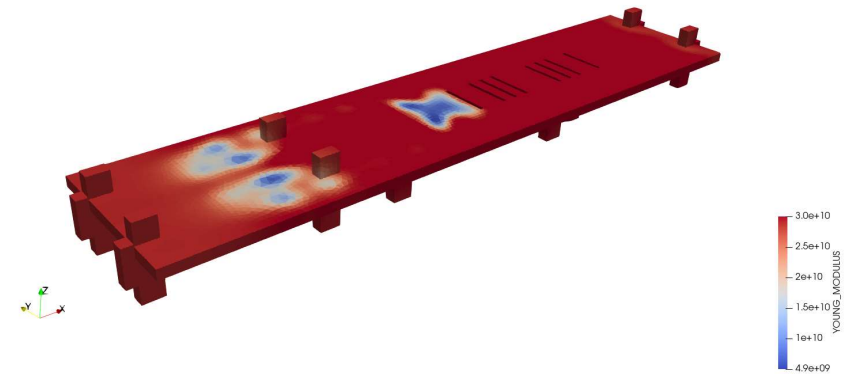


Concrete Bridge

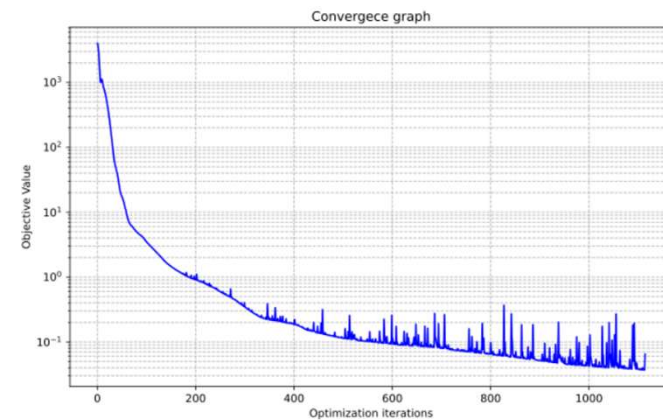
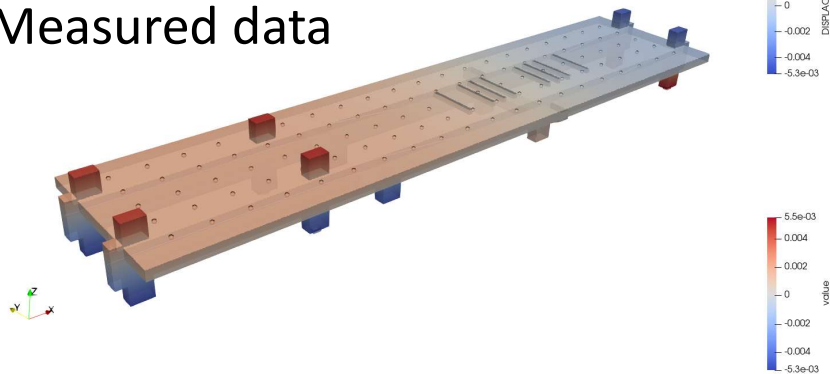
Damaged system



Found Damage



Measured data

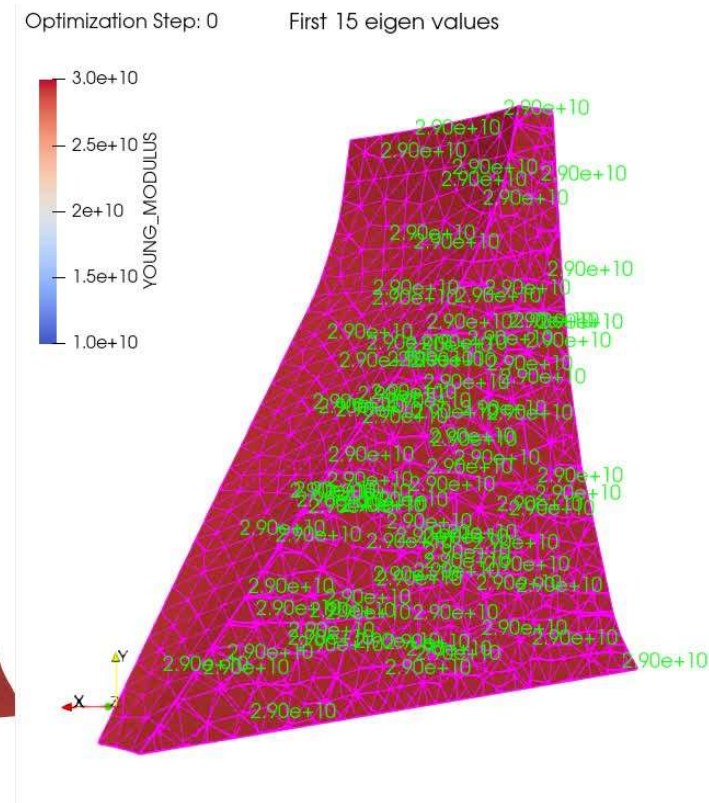
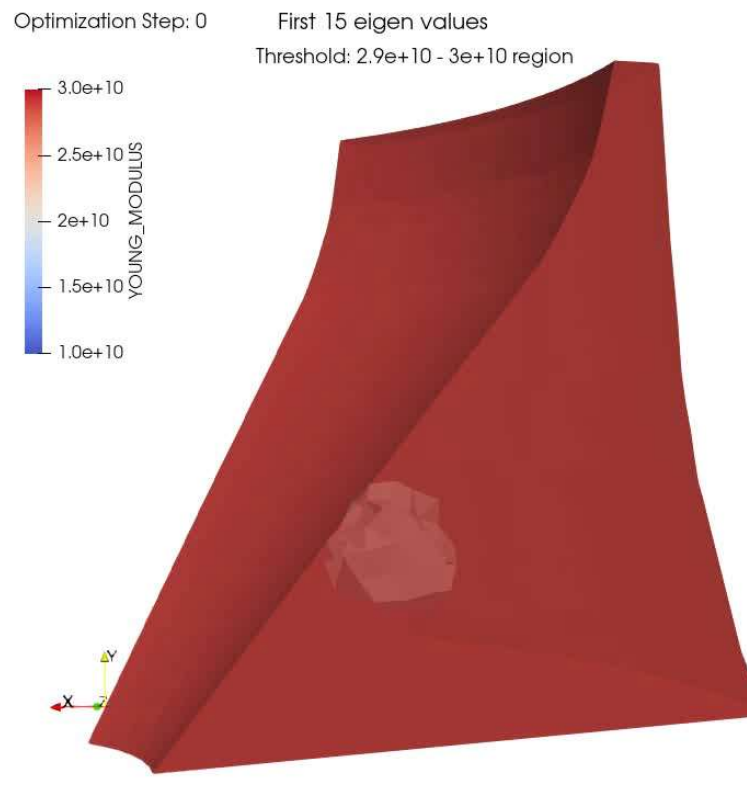


6/11/2024 – 8/11/2024

Kratos Workshop 2024 - Padova

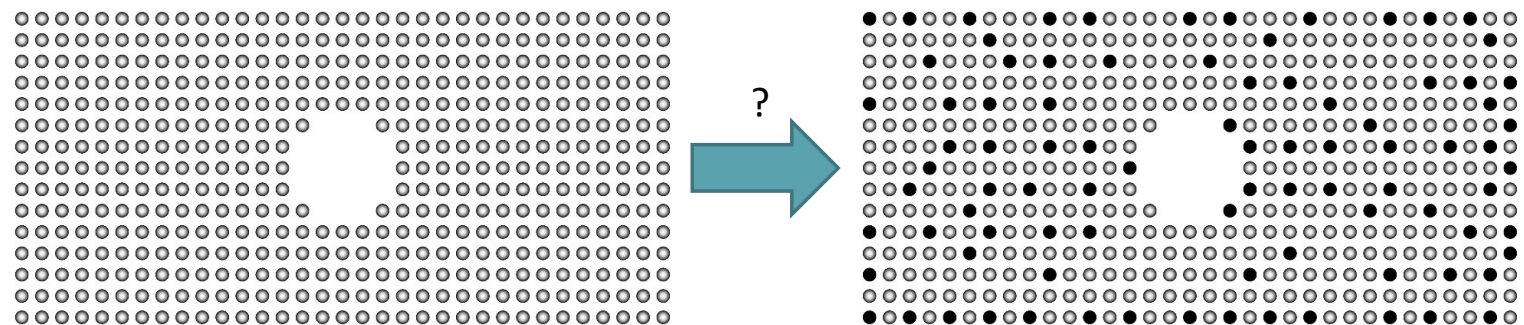


Dam example: Matching Eigen values



Sensor Placement Algorithms – Continuous Optimization

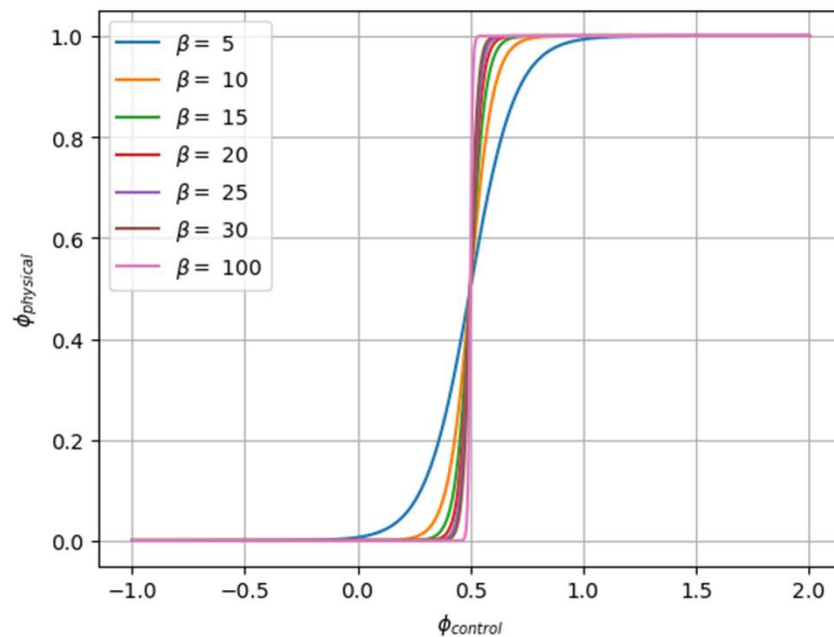
- We need differentiable sensor status “active”, “inactive” variable.
- We need differentiable objective/cost functions for
 - Coverage ratio.
 - Largest cluster size ratio.
 - Minimum distance between sensors.
 - Number of sensors.
 - Etc...



Sensor Placement Algorithms – Continuous Optimization

- We need differentiable sensor status “active”, “inactive” variable...

$$\phi_{physical} = \frac{1}{1 + e^{-2\beta(\phi_{control} - 0.5)}}$$



$\phi_{physical} = 1$ ➡ Sensor is used for system identification

$\phi_{physical} = 0$ ➡ Sensor is **not** used for system identification



Response functions

- Minimum distance between sensors objective/cost function:

$$J_{distance} = \left[\sum_i^M \sum_{j=i+1}^M \phi_{physical,i} \phi_{physical,j} (d_{ij})^{-p} \right]^{-\frac{1}{p}}$$

- Largest cluster size ratio objective/cost function:

$$J_{localization} = \frac{1}{\sum_j^N |\Omega|_j} \left(\sum_j^N Cluster_j^p \right)^{\frac{1}{p}}$$

- Coverage ratio objective/cost function:

$$J_{coverage} = \frac{1}{\sum_j^N |\Omega|_j} \sum_j^N |\Omega|_j \begin{cases} 1 & \text{if } \exists m_{ij} = 1 \forall i \in [1, M] \\ 0 & \text{otherwise} \end{cases}$$



Sensor Placement Algorithms – Continuous Optimization

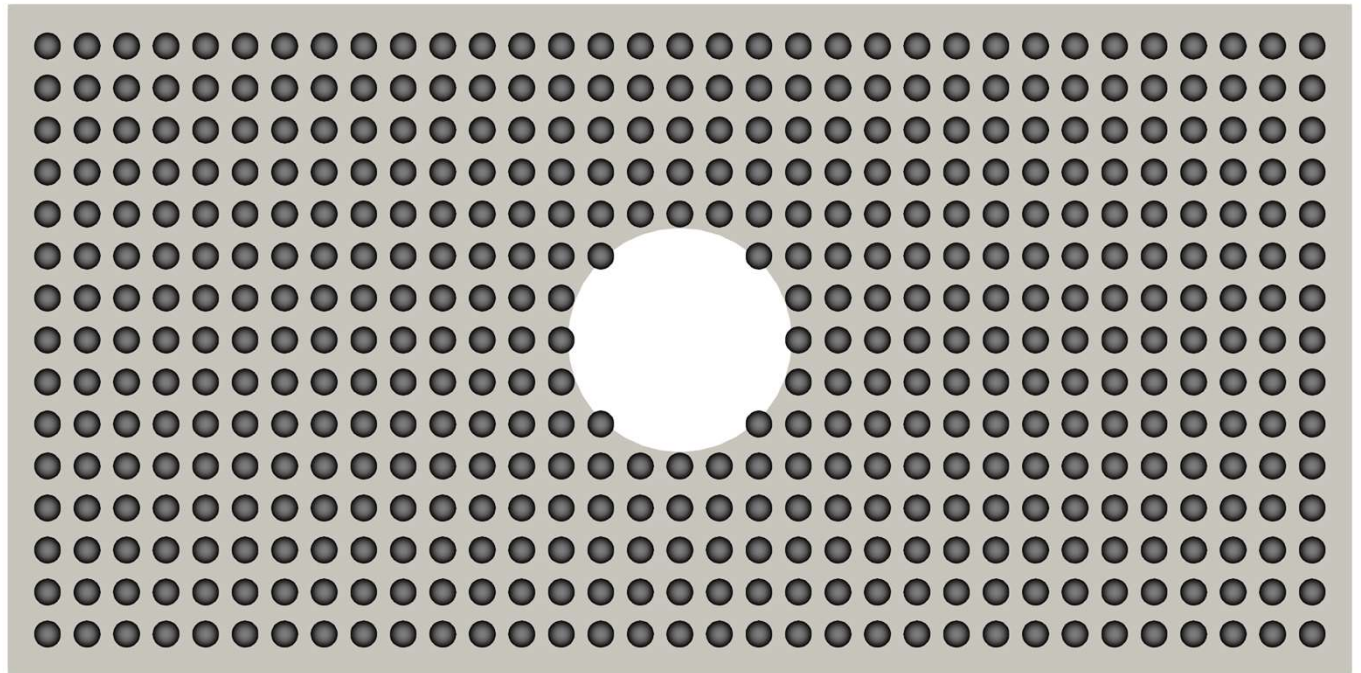
- Optimization problem statement:

$$\max_{\phi_{physical}} J_{distance}$$

s.t.

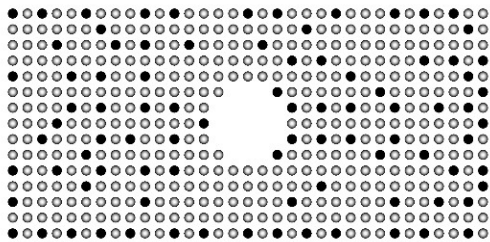
$$J_{localization} \leq 0.02$$

$$J_{covarage} \geq 0.99$$

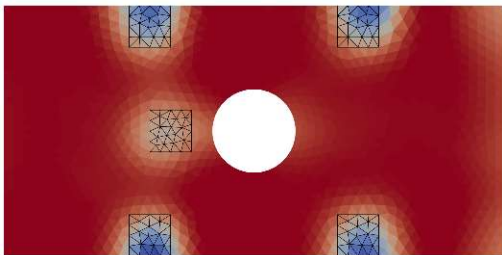


Results – Plate with Whole – Localization Comparison

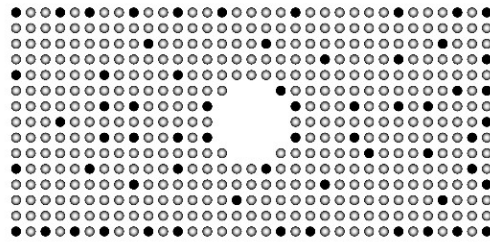
Localization = 2%



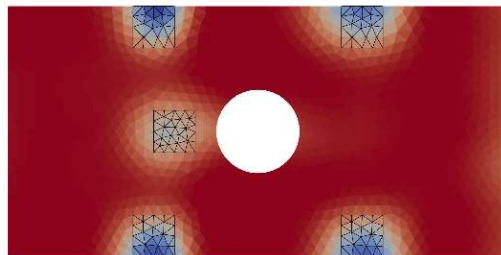
89



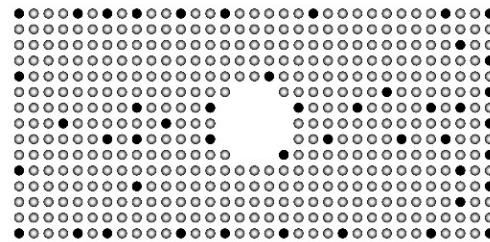
Localization = 3%



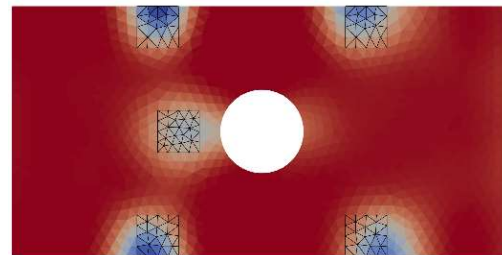
62



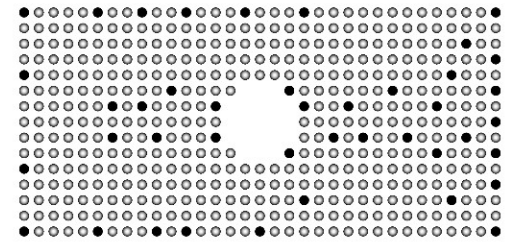
Localization = 4%



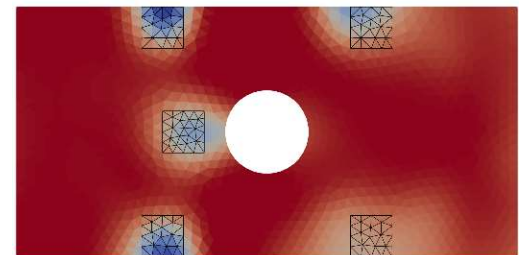
47



Localization = 5%



42



Summary

- Optimization Application is used to formulate and solve SI problem
- New sensor types and Response functions can be implemented
- Various SI parameters can be added and controlled
- And more ...



Main Developers



Suneth Warnakulasuriya
suneth.warnakulasuriyal@tum.de



Ihar Antonau
ihar.antonau@tu-braunschweig.de



Talhah Shamshad Ali Ansari
talhah.ansari@tum.de



Reza Najian Asl





 <https://github.com/KratosMultiphysics>

 www.youtube.com/@kratosmultiphysics3578

